COSC2637 - Big Data Processing

Lecture 11 – problem solving

Spark Streaming 2

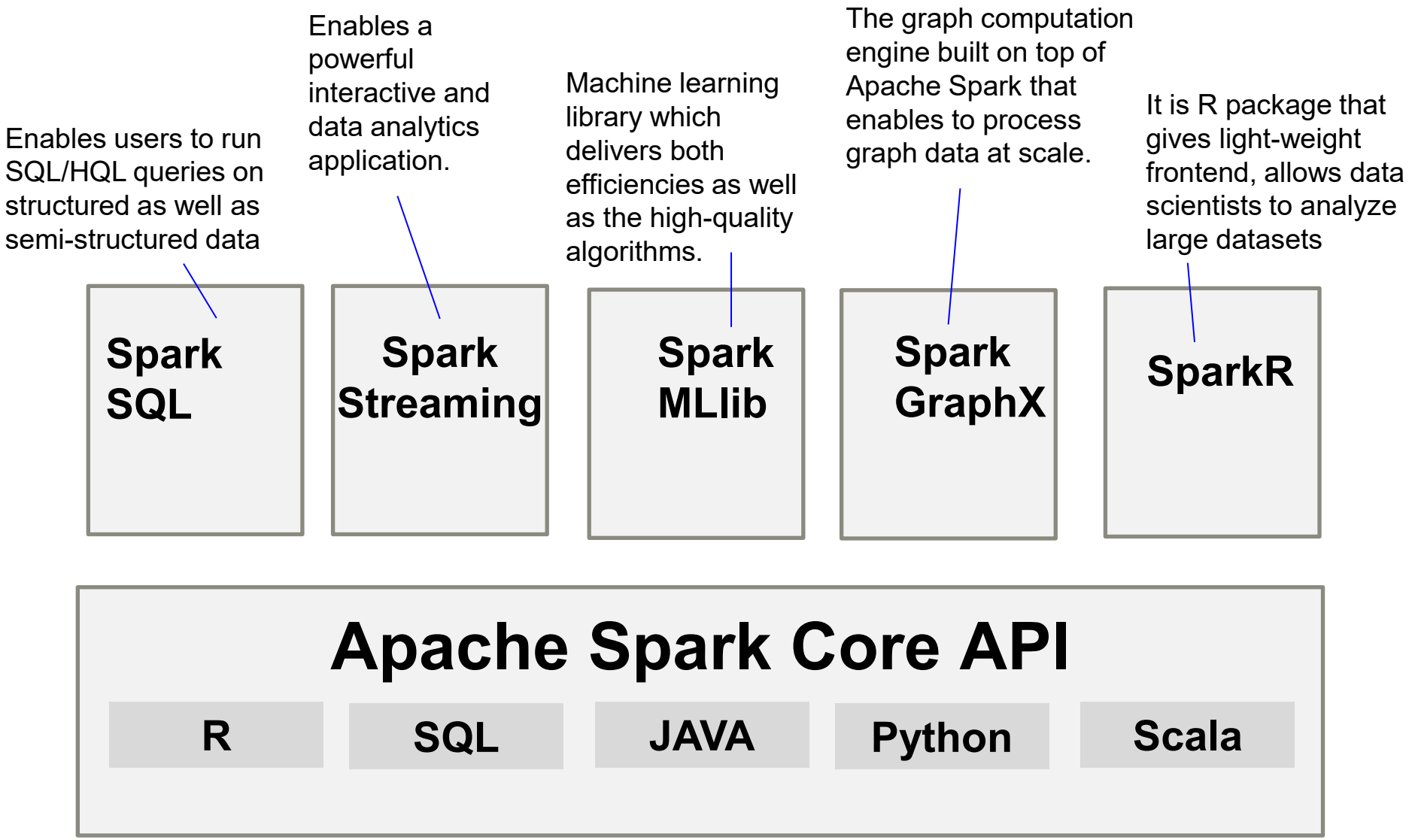
RMIT UNIVERSITY

RMIT Classification: Trusted

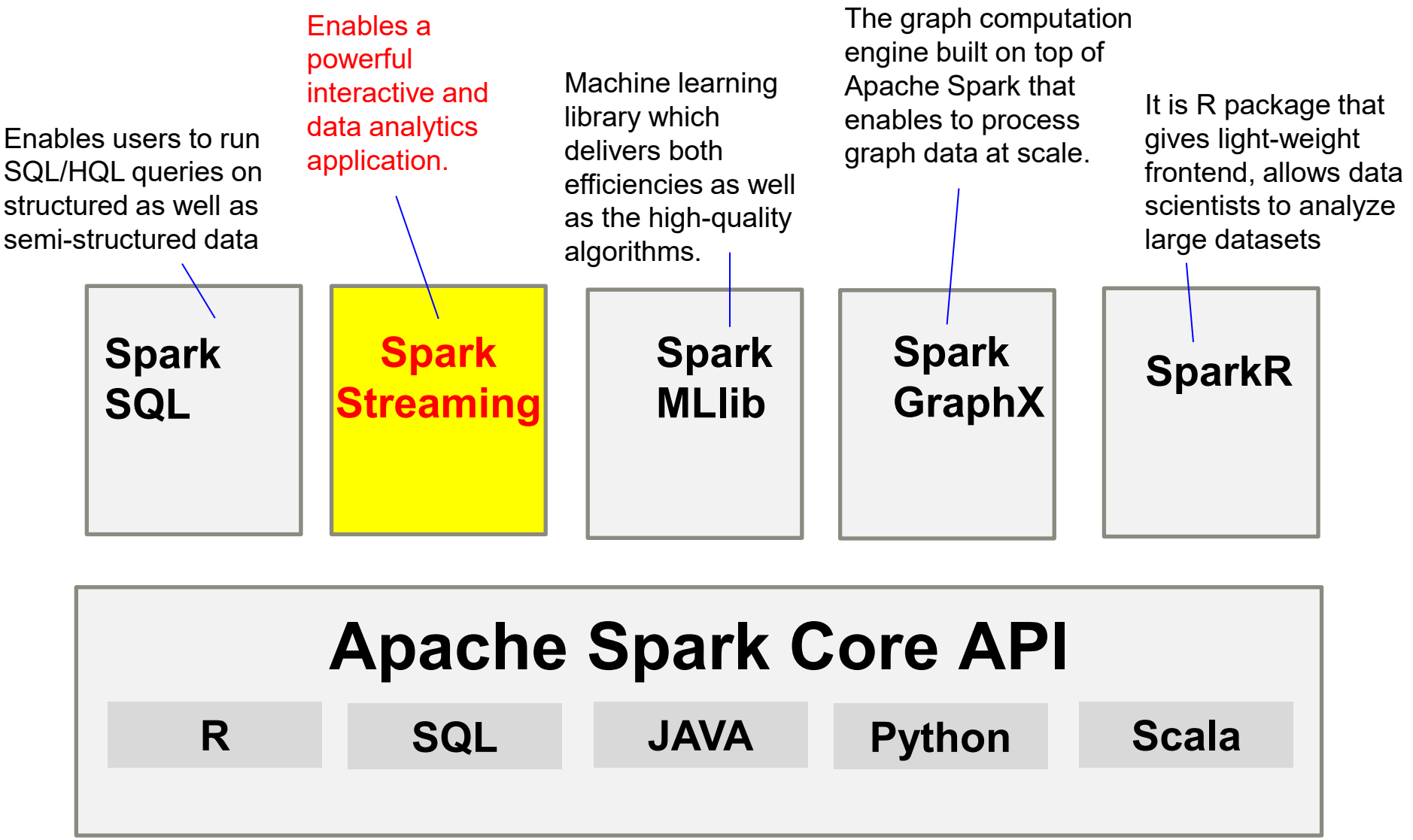
Q1 - Regarding Spark Streaming inputs and outputs, which statement is correct.

- HDFS can only be used as the input source of Spark streaming since it is reliable storage.
- Socket connections are a type of advanced source of spark streaming and require linking against extra dependencies
- The output of spark streaming can be a live dashboard from which the end user can monitor states in real time.
- Kafka is available in the StreamingContext API.

Apache Spark Ecosystem



Apache Spark Ecosystem



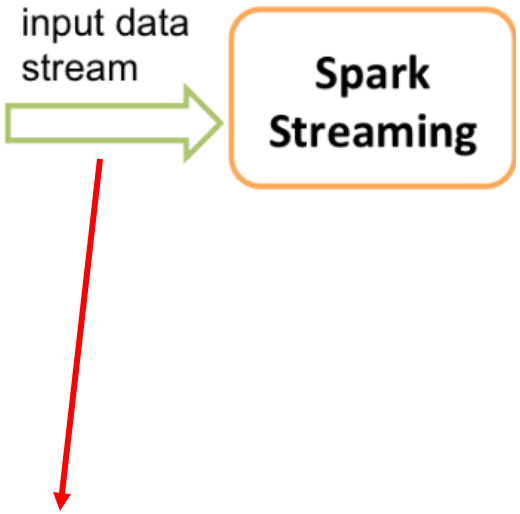
DStreams – Real-time RDDs

- Spark Streaming - DStreams



DStreams – Real-time RDDs

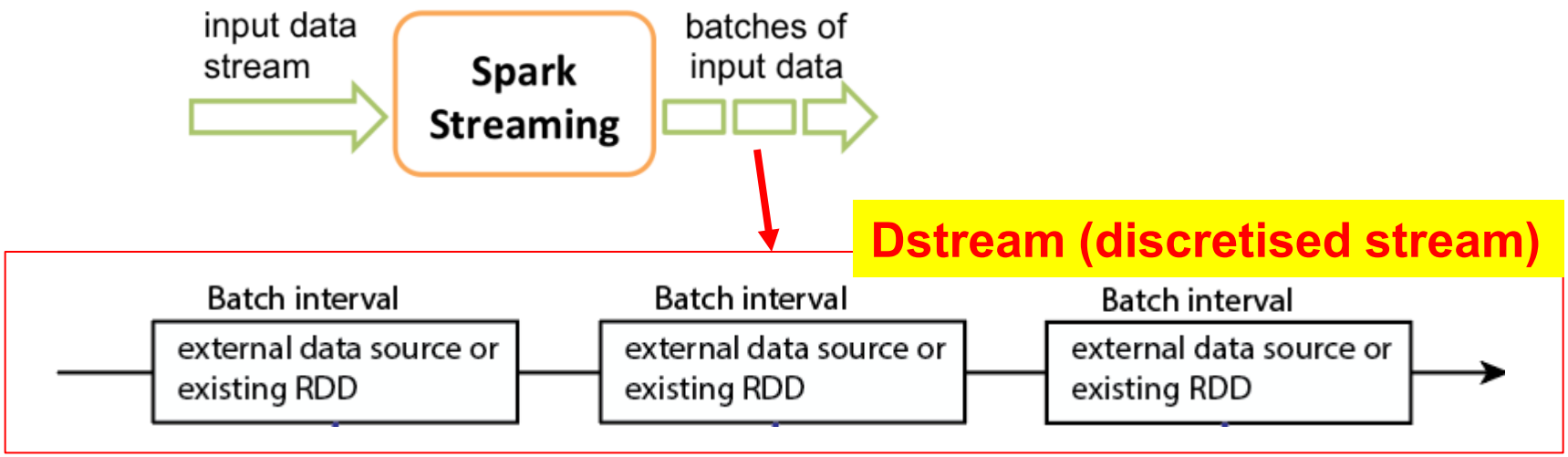
- Spark Streaming - DStreams



An endless flow of input data

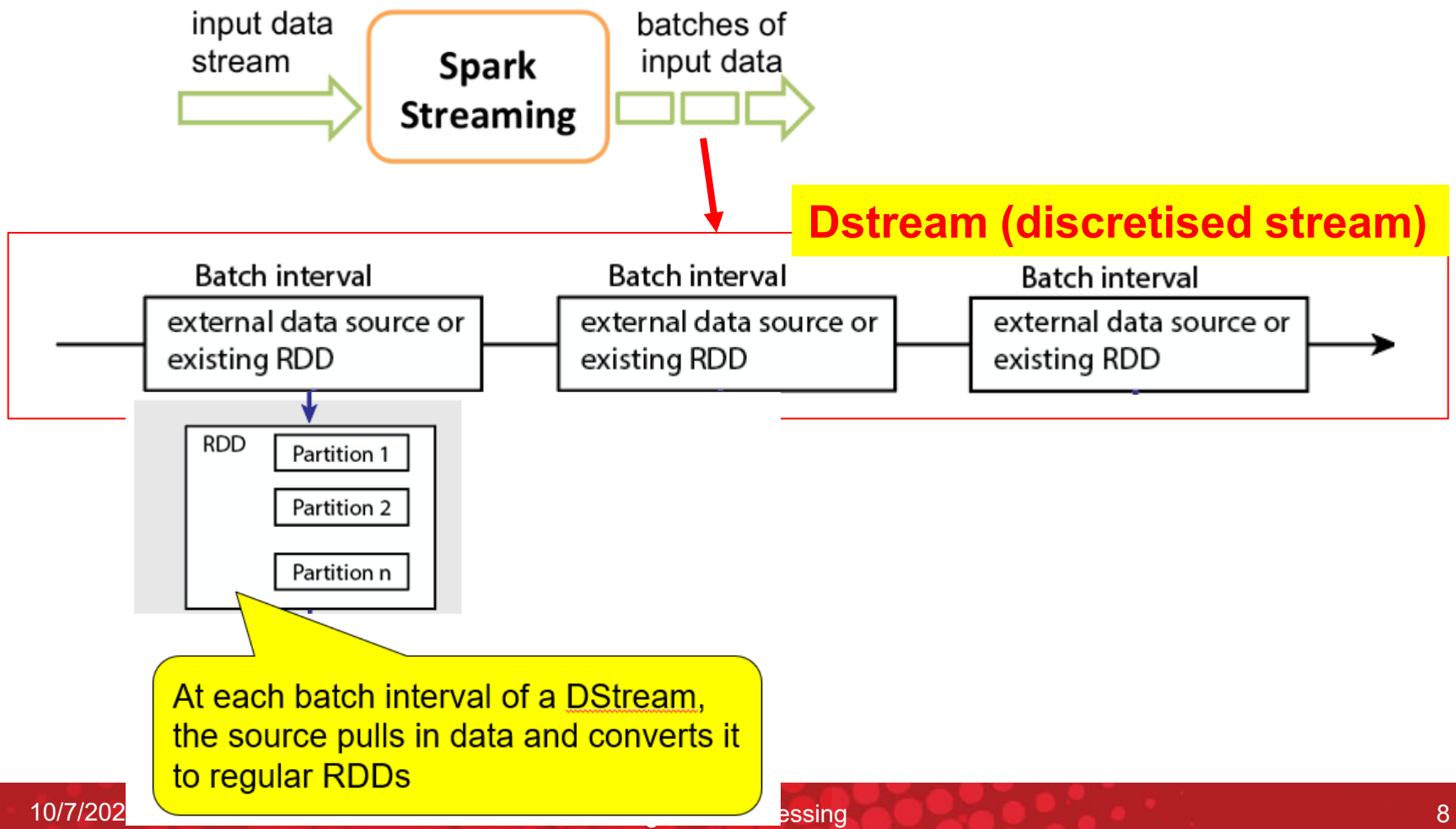
DStreams – Real-time RDDs

- Spark Streaming - DStreams



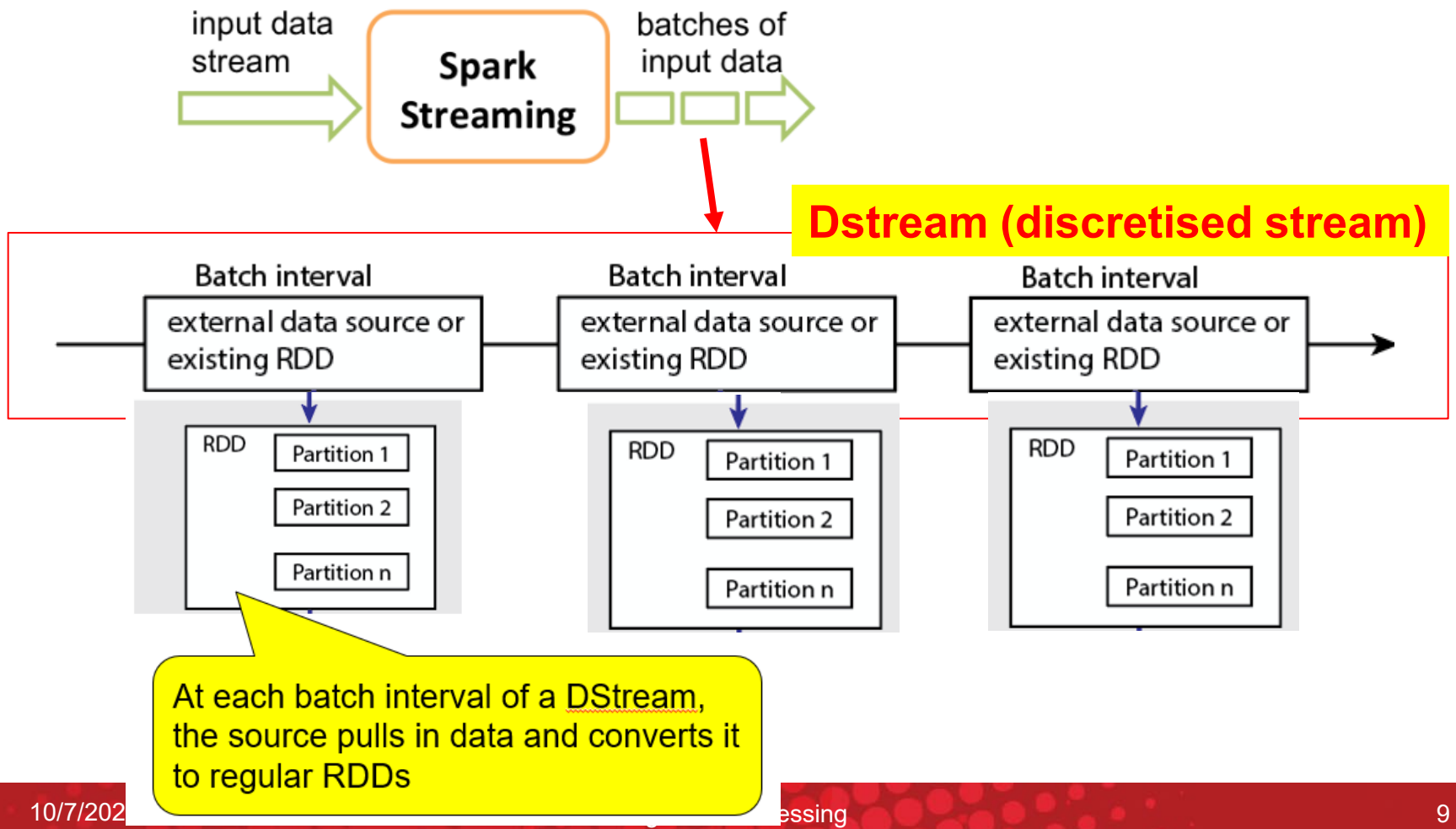
DStreams – Real-time RDDs

- Spark Streaming - DStreams



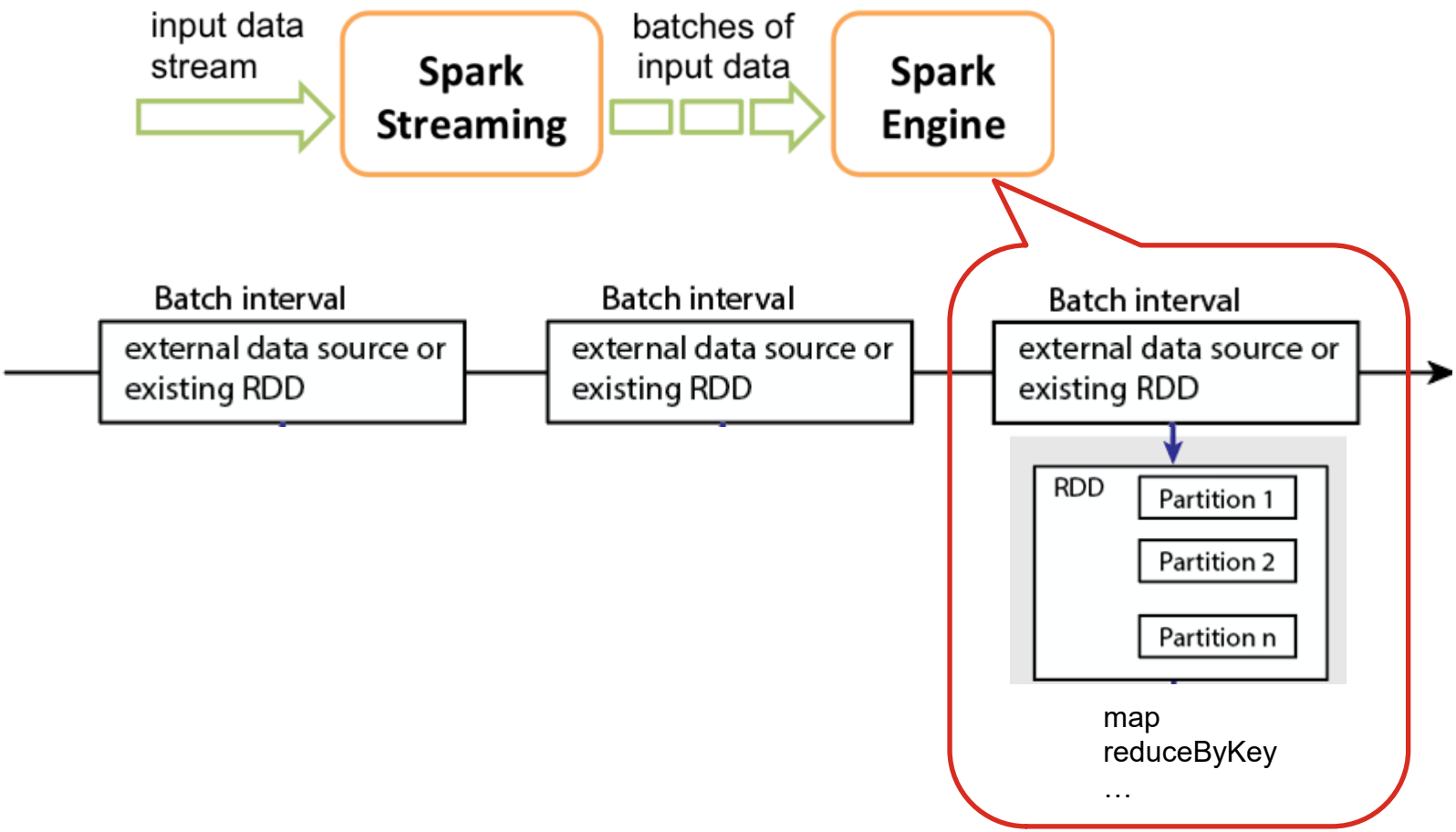
DStreams – Real-time RDDs

- Spark Streaming - DStreams



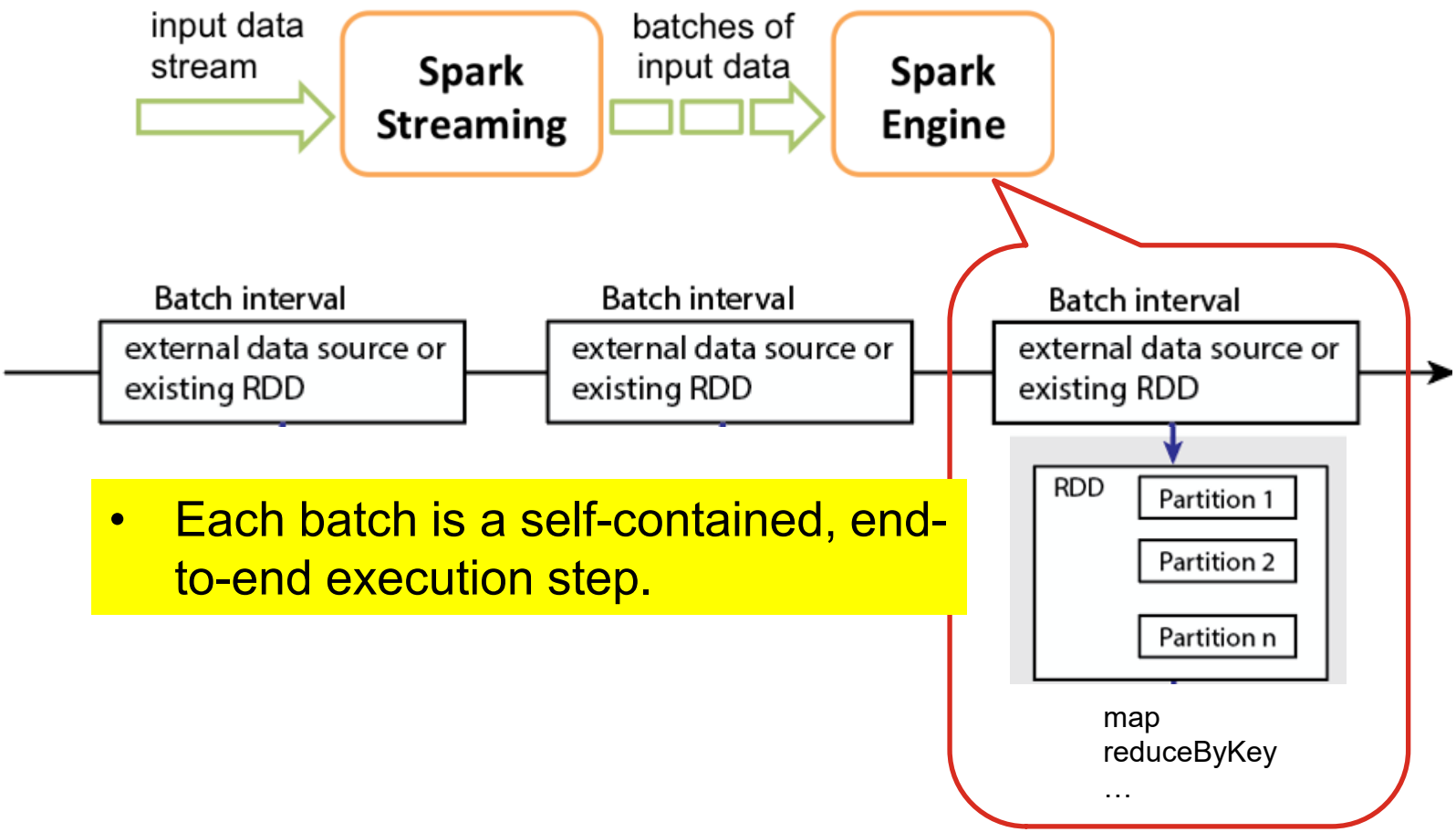
DStreams – Real-time RDDs

- Spark Streaming - DStreams



DStreams – Real-time RDDs

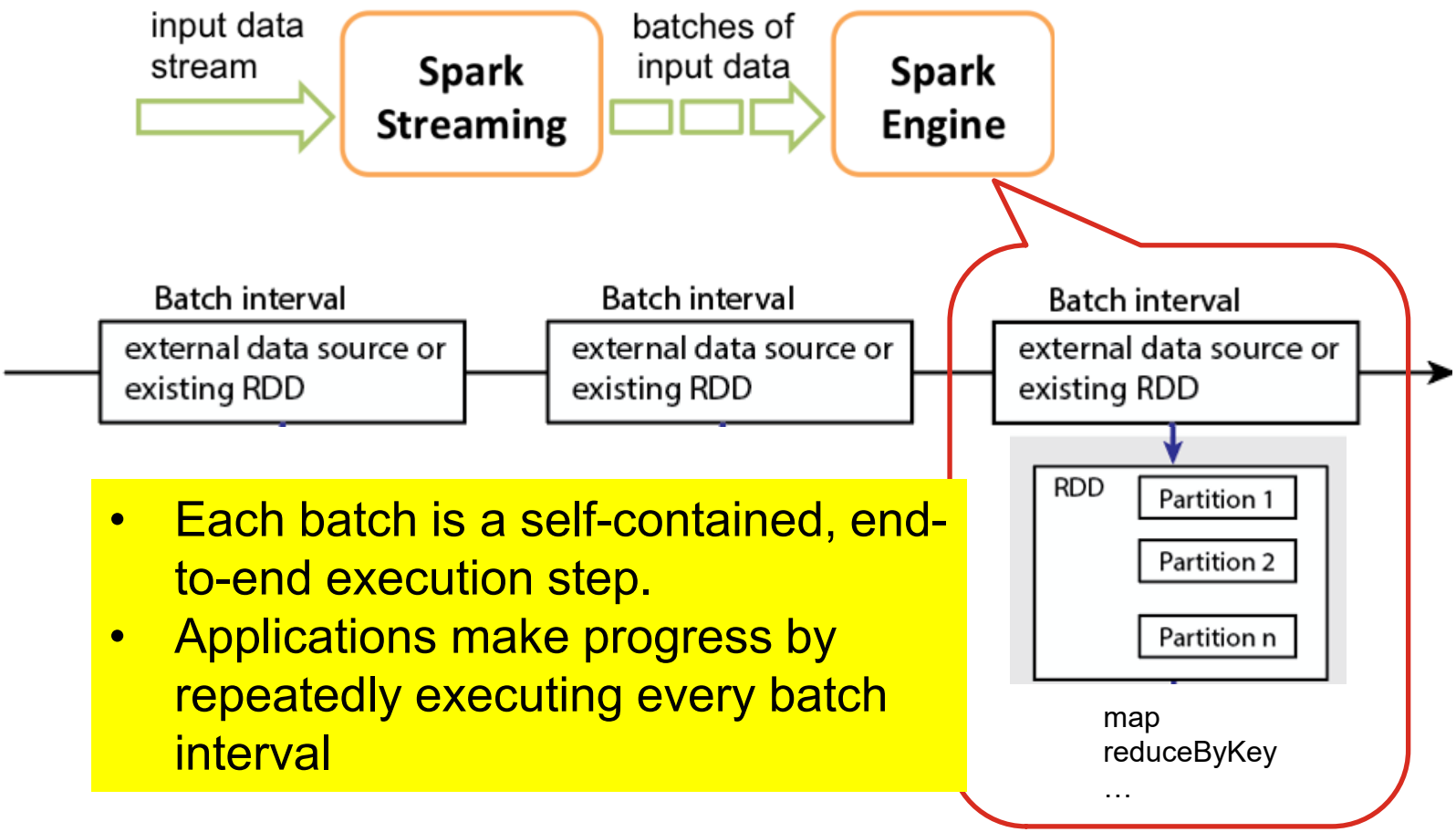
- Spark Streaming - DStreams



- Each batch is a self-contained, end-to-end execution step.

DStreams – Real-time RDDs

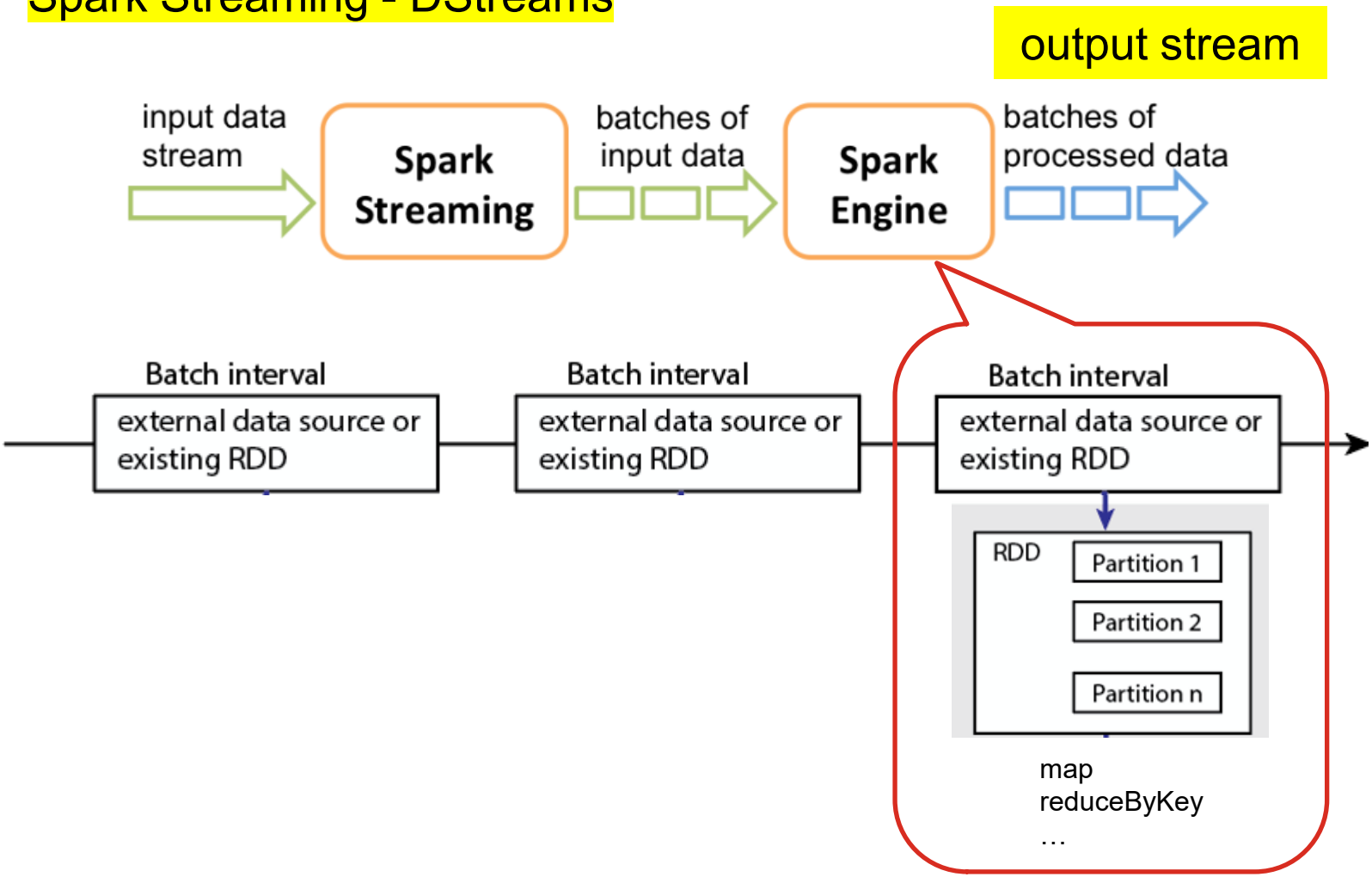
- Spark Streaming - DStreams



- Each batch is a self-contained, end-to-end execution step.
- Applications make progress by repeatedly executing every batch interval

DStreams – Real-time RDDs

- Spark Streaming - DStreams



Spark Streaming Input and Output

Spark Streaming

- inputs from various reliable inputs sources like Flume, HDFS, and Kafka etc.
- outputs the processed data to filesystems, database or live dashboards.



Spark Streaming Input and Output

Spark Streaming

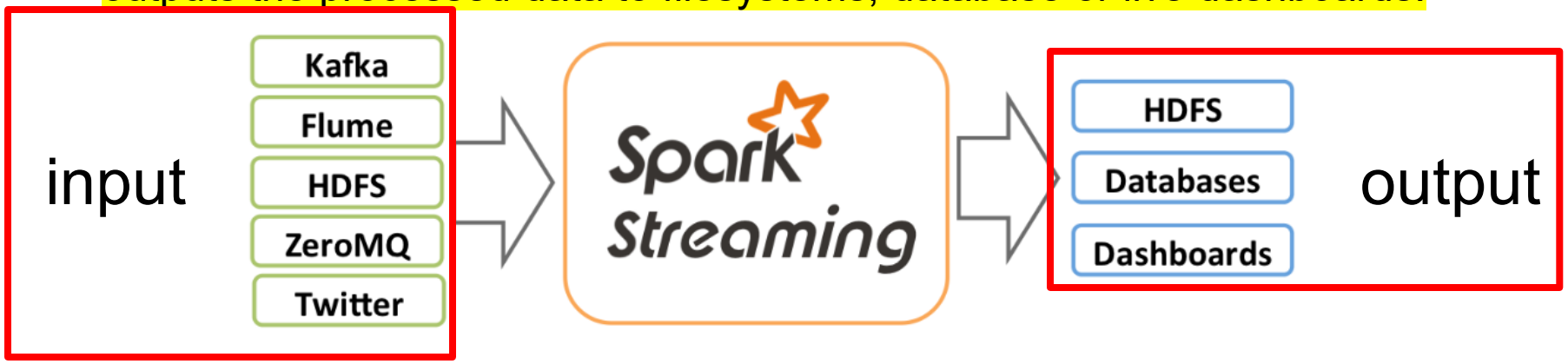
- inputs from various reliable inputs sources like Flume, HDFS, and Kafka etc.
- outputs the processed data to filesystems, database or live dashboards.



Spark Streaming Input and Output

Spark Streaming

- inputs from various reliable inputs sources like Flume, HDFS, and Kafka etc.
- outputs the processed data to filesystems, database or live dashboards.



Spark Streaming Input

Spark Streaming provides two categories of built-in streaming input sources:

- ***Basic sources*** **–** These sources are directly available in the *StreamingContext API*.

  Examples: file systems, and socket connections

Spark Streaming Input

Spark Streaming provides two categories of built-in streaming input sources:

- ***Basic sources –*** These sources are directly available in the *StreamingContext API*.

  Examples: file systems, and socket connections

- ***Advanced Sources –*** These sources are available by extra utility classes like *Kafka, Flume, Kinesis.* Thus, requires linking against extra dependencies.

  For example:
  Kafka: the artifact required for Kafka is *spark-streaming-kafka-0-8_2.11.*
  Flume: the artifact requires for Flume is *dspark-streaming-flume_2.11.*
  Kinesis: the artifact required for Kinesis is *spark-streaming-kinesis-asl_2.11.*

RMIT Classification: Trusted

Q1 - Regarding Spark Streaming inputs and outputs, which statement is correct.

- HDFS can only be used as the input source of Spark streaming since it is reliable storage.
- Socket connections are a type of advanced source of spark streaming and require linking against extra dependencies
- The output of spark streaming can be a live dashboard from which the end user can monitor states in real time.
- Kafka is available in the StreamingContext API.

RMIT Classification: Trusted

Q1 - Regarding Spark Streaming inputs and outputs, which statement is correct.



HDFS can only be used as the input source of Spark streaming since it is reliable storage.

- Socket connections are a type of advanced source of spark streaming and require linking against extra dependencies
- The output of spark streaming can be a live dashboard from which the end user can monitor states in real time.
- Kafka is available in the StreamingContext API.

Q1 - Regarding Spark Streaming inputs and outputs, which statement is correct.

- ✗ HDFS can only be used as the input source of Spark streaming since it is reliable storage.
- ✗ Socket connections are a type of advanced source of spark streaming and require linking against extra dependencies
- ○ The output of spark streaming can be a live dashboard from which the end user can monitor states in real time.
- ○ Kafka is available in the StreamingContext API.

Q1 - Regarding Spark Streaming inputs and outputs, which statement is correct.



HDFS can only be used as the input source of Spark streaming since it is reliable storage.



Socket connections are a type of advanced source of spark streaming and require linking against extra dependencies



The output of spark streaming can be a live dashboard from which the end user can monitor states in real time.

- Kafka is available in the StreamingContext API.

Q1 - Regarding Spark Streaming inputs and outputs, which statement is correct.



HDFS can only be used as the input source of Spark streaming since it is reliable storage.



Socket connections are a type of advanced source of spark streaming and require linking against extra dependencies



The output of spark streaming can be a live dashboard from which the end user can monitor states in real time.



Kafka is available in the StreamingContext API.

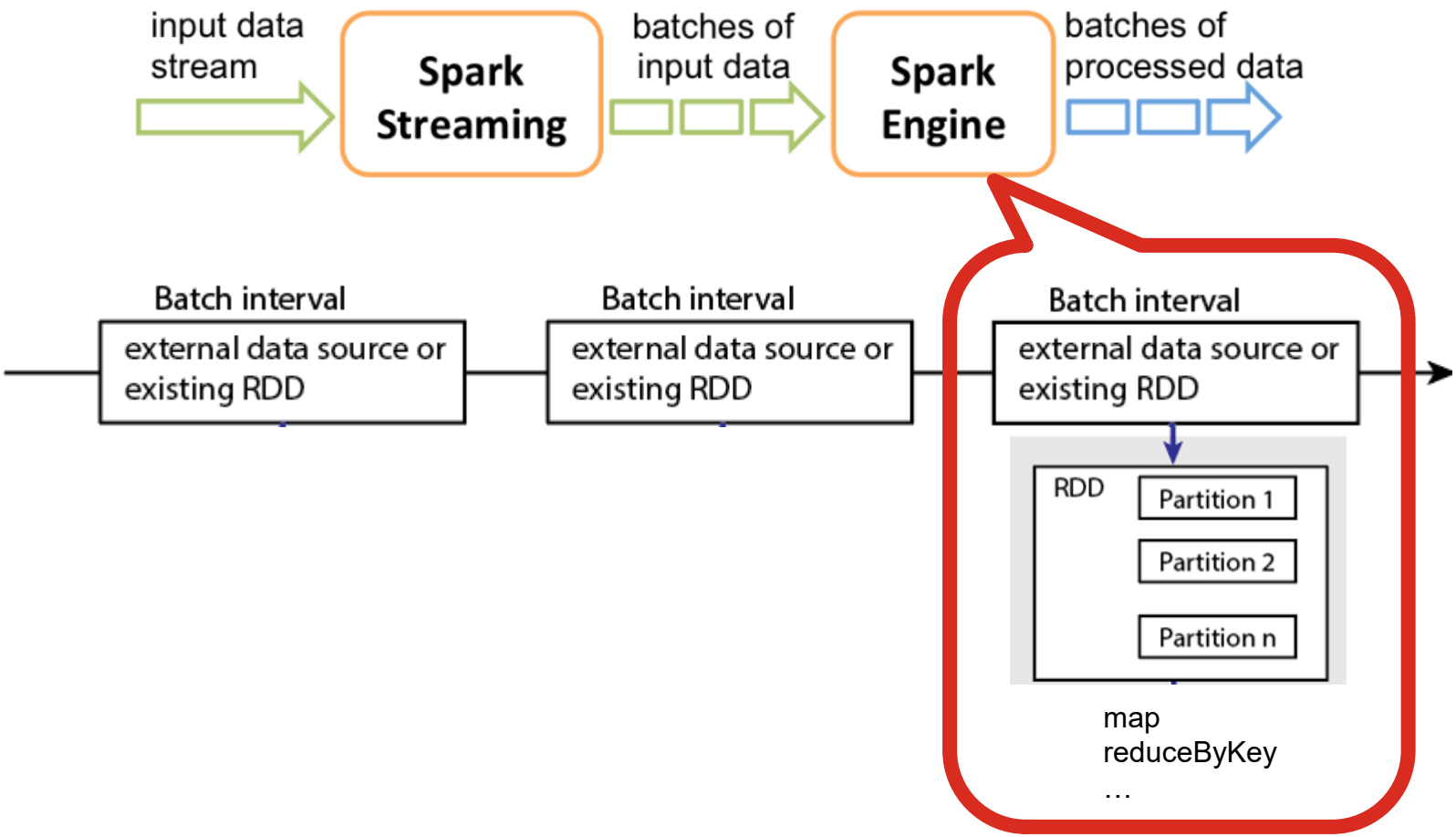
```
val runningCounts = pairs.updateStateByKey[Int](updateFunction )

def updateFunction
        (newValues: Seq[Int], runningCount: Option[Int]): Option[Int] = {
            // add the new values with the previous running count to get the new count
            val newCount = ...
            Some(newCount)
}
```

Q2 - What are information represented by newValues and runningCount respectively?

- newValues having a sequence of values for each key in new RDD
- newValue will be the latest value for each key during stream data processing.
- runningCount is the aggregation of values over the current stream RDD batch.
- runningCount having the count for each key of the previous RDDs

Spark Streaming Execution



DStream Operations

Transformation

- Map
- flatMap
- Fliter
- Repartition
- Union
- Count
- Reduce
- countByValue
- reduceByKey
- Join
- Cogroup
- Transform

DStream Operations

Transformation

- Map
- flatMap
- Fliter
- Repartition
- Union
- Count
- Reduce
- countByValue
- reduceByKey
- Join
- Cogroup
- Transform

Stateful Transformation

- updateStateByKey

maintains per-key state across batches until explicitly removed.

- Window Operations

retains data for a bounded time window

  - Window
  - countByWindow
  - reduceByWindow
  - reduceByKeyAndWindow

DStream Operations

Transformation

- Map
- flatMap
- Fliter
- Repartition
- Union
- Count
- Reduce
- countByValue
- reduceByKey
- Join
- Cogroup
- Transform

Stateful Transformation

- updateStateByKey

maintains per-key state across batches until explicitly removed.

- Window Operations

retains data for a bounded time window

  - Window
  - countByWindow
  - reduceByWindow
  - reduceByKeyAndWindow

Action

- Output Operations
  - print()
  - saveAsTextFiles()
  - saveAsObjectFiles()
  - saveAsHadoopFiles()
  - foreachRDD(func)

Transformations on DStreams

Similar to that of RDDs, transformations allow the data from the input DStream to be modified. DStreams support many transformations available on normal Spark RDD's.

Transformation	Meaning
map(*func*)	Return a new DStream by passing each element of the source DStream through a function *func*.
flatMap(*func*)	Similar to map, but each input item can be mapped to 0 or more output items.
filter(*func*)	Return a new DStream by selecting only the records of the source DStream on which *func* returns true.
repartition(*numPartitions*)	Changes the level of parallelism in this DStream by creating more or fewer partitions.
union(*otherStream*)	Return a new DStream that contains the union of the elements in the source DStream and *otherDStream*.
count()	Return a new DStream of single-element RDDs by counting the number of elements in each RDD of the source DStream.

Transformations on DStreams

Similar to that of RDDs, transformations allow the data from the input DStream to be modified. DStreams support many transformations available on normal Spark RDD's.

Transformation	Meaning
map(*func*)	Return a new DStream by passing each elemen *func*.
flatMap(*func*)	Similar to map, but each input item can be mapp
filter(*func*)	Return a new DStream by selecting only the rec returns true.
repartition(*numPartitions*)	Changes the level of parallelism in this DStream
union(*otherStream*)	Return a new DStream that contains the union o *otherDStream*.
count()	Return a new DStream of single-element RDDs each RDD of the source DStream.

Transformation

- Map
- flatMap
- Fliter
- Repartition
- Union
- Count
- Reduce
- countByValue
- reduceByKey
- Join
- Cogroup
- Transform

Transformations on Dstreams-
UpdateStateByKey Operation

It also allows you to maintain arbitrary state which is continuously updated with new information.

- To use this, you will have to do two steps.

Stateful Transformation

- updateStateByKey

maintains per-key state across batches until explicitly removed.

- Window Operations

retains data for a bounded time window
  - Window
  - countByWindow
  - reduceByWindow
  - reduceByKeyAndWindow

RMIT Classification: Trusted

Transformations on Dstreams-
UpdateStateByKey Operation

It also allows you to maintain arbitrary state which is continuously updated with new information.

- To use this, you will have to do two steps.
  1. Define the state - The state can be an arbitrary data type.

RMIT Classification: Trusted

Transformations on Dstreams-
UpdateStateByKey Operation

It also allows you to maintain arbitrary state which is continuously updated with new information.

- To use this, you will have to do two steps.
  1. Define the state - The state can be an arbitrary data type.
  2. Define the state update function - Specify with a function how to update the state using the previous state and the new values from an input stream.
     - In every batch, Spark will apply the state update function for all existing keys, regardless of whether they have new data in a batch or not.
     - If the update function returns ”None” then the key-value pair will be eliminated.

RMIT Classification: Trusted

Transformations on Dstreams-
UpdateStateByKey Operation

It also allows you to maintain arbitrary state which is continuously updated with new information.

- To use this, you will have to do two steps.
  1. Define the state - The state can be an arbitrary data type.
  2. Define the state update function - Specify with a function how to update the state using the previous state and the new values from an input stream.
     - In every batch, Spark will apply the state update function for all existing keys, regardless of whether they have new data in a batch or not.
     - If the update function returns ”None” then the key-value pair will be eliminated.

Transformations on Dstreams-
UpdateStateByKey Operation

It also allows you to maintain arbitrary state which is continuously updated with new information.

- To use this, you will have to do two steps.
  1. Define the state - The state can be an arbitrary data type.
  2. Define the state update function - Specify with a function how to update the state using the previous state and the new values from an input stream.
     - In every batch, Spark will apply the state update function for all existing keys, regardless of whether they have new data in a batch or not.
     - If the update function returns ”None” then the key-value pair will be eliminated.

RMIT Classification: Trusted

Transformations on Dstreams-
UpdateStateByKey Operation

An example

You want to maintain a running count of each word seen in a text data stream. Here, the running count is the state and it is an integer.

Transformations on Dstreams-
UpdateStateByKey Operation

An example

You want to maintain a running count of each word seen in a text data stream. Here, the running count is the state and it is an integer. We define the update function as:

```
def updateFunction(newValues: Seq[Int], runningCount: Option[Int]): Option[Int] = {
        // add the new values with the previous running count to get the new count
        val newCount = ...
        Some(newCount)
}
```

Transformations on Dstreams-

UpdateStateByKey Operation

An example

You want to maintain a running count of each word seen in a text data stream. Here, the running count is the state and it is an integer. We define the update function as:

```
def updateFunction(newValues: Seq[Int], runningCount: Option[Int]): Option[Int] = {
        // add the new values with the previous running count to get the new count
        val newCount = ...
        Some(newCount)
}
```

Apply the function on a Dstream pairs which contains (word, 1) pairs

```
val runningCounts = pairs.updateStateByKey[Int](updateFunction _)
```

Transformations on Dstreams-

UpdateStateByKey Operation

An example

You want to maintain a running count of each word seen in a text data stream. Here, the running count is the state and it is an integer. We define the update function as:

```
def updateFunction(newValues: Seq[Int], runningCount: Option[Int]): Option[Int] = {
        // add the new values with the previous running count to get the new count
        val newCount = ...
        Some(newCount)
}
```

Apply the function on a Dstream pairs which contains (word, 1) pairs

```
val runningCounts = pairs.updateStateByKey[Int](updateFunction _)
```

Transformations on Dstreams-
UpdateStateByKey Operation

An example

You want to maintain a running count of each word seen in a text data stream. Here, the running count is the state and it is an integer. We define the update function as:

```
def updateFunction(newValues: Seq[Int], runningCount: Option[Int]): Option[Int] = {
        // add the new values with the previous running count to get the new count
        val newCount = ...
        Some(newCount)
}
```

Apply the function on a Dstream pairs which contains (word, 1) pairs

```
val runningCounts = pairs.updateStateByKey[Int](updateFunction _)
```

Transformations on Dstreams-
UpdateStateByKey Operation

An example

You want to maintain a running count of each word seen in a text data stream. Here, the running count is the state and it is an integer. We define the update function as:

```
def updateFunction(newValues: Seq[Int], runningCount: Option[Int]): Option[Int] = {
        // add the new values with the previous running count to get the new count
        val newCount = ...
        Some(newCount)
}
```

Apply the function on a Dstream pairs which contains (word, 1) pairs

```
val runningCounts = pairs.updateStateByKey[Int](updateFunction _)
```

The update function will be called for each word

- with *newValues* having a sequence of 1's for each key in new RDD
- the *runningCount* having the count for each key of the previous RDDs.

Transformations on Dstreams-
UpdateStateByKey Operation

An example

You want to maintain a running count of each word seen in a text data stream. Here, the running count is the state and it is an integer. We define the update function as:

```
def updateFunction(newValues: Seq[Int], runningCount: Option[Int]): Option[Int] = {
        // add the new values with the previous running count to get the new count
        val newCount = ...
        Some(newCount)
}
```

Apply the function on a Dstream pairs which contains (word, 1) pairs

```
val runningCounts = pairs.updateStateByKey[Int](updateFunction _)
```

The update function will be called for each word

- with *newValues* having a sequence of 1's for each key in new RDD
- the *runningCount* having the count for each key of the previous RDDs.

Transformations on Dstreams-
UpdateStateByKey Operation

An example

You want to maintain a running count of each word seen in a text data stream. Here, the running count is the state and it is an integer. We define the update function as:

```
def updateFunction(newValues: Seq[Int], runningCount: Option[Int]): Option[Int] = {
        // add the new values with the previous running count to get the new count
        val newCount = ...
        Some(newCount)
}
```

Apply the function on a Dstream pairs which contains (word, 1) pairs

```
val runningCounts = pairs.updateStateByKey[Int](updateFunction _)
```

The update function will be called for each word

- with *newValues* having a sequence of 1's for each key in new RDD
- the *runningCount* having the count for each key of the previous RDDs.

Note that using *updateStateByKey* requires the checkpoint directory to be configured

Transformations on Dstreams-
UpdateStateByKey Operation

```
import org.apache.spark._
import org.apache.spark.streaming._

object StatefulNetworkWordCount {
  def updateFunction(newValues: Seq[Int], runningCount: Option[Int]): Option[Int] = {
    val newCount = runningCount.getOrElse(0) + newValues.sum
    Some(newCount)
  }

  def main(args: Array[String]) {
    val conf = new SparkConf().setMaster("local[8]").setAppName("StatefulNetworkWordCount")
    val ssc = new StreamingContext(conf, Seconds(5))
    ssc.checkpoint(".")
    val lines = ssc.socketTextStream("localhost", 9999)
    val words = lines.flatMap(_.split(" "))
    val pairs = words.map(word => (word, 1))
    val runningCounts = pairs.updateStateByKey[Int](updateFunction _)
    runningCounts.print()
    ssc.start() // Start the computation
    ssc.awaitTermination() // Wait for the computation to terminate
  }
}
```

Transformations on Dstreams-
UpdateStateByKey Operation

- Create a local StreamingContext with 8 working threads (e.g., 2 cores) and batch interval of 5 seconds. local[*] : Run Spark locally with as many working threads as logical cores on your machine.

```
import org.apache.spark._
import org.apache.spark.streaming._

object StatefulNetworkWordCount {
  def updateFunction(newValues: Seq[Int], runningCount: Option[Int]): Option[Int] = {
    val newCount = runningCount.getOrElse(0) + newValues.sum
    Some(newCount)
  }

  def main(args: Array[String]) {
    val conf = new SparkConf().setMaster("local[8]").setAppName("StatefulNetworkWordCount")
    val ssc = new StreamingContext(conf, Seconds(5))
    ssc.checkpoint(".")
    val lines = ssc.socketTextStream("localhost", 9999)
    val words = lines.flatMap(_.split(" "))
    val pairs = words.map(word => (word, 1))
    val runningCounts = pairs.updateStateByKey[Int](updateFunction _)
    runningCounts.print()
    ssc.start() // Start the computation
    ssc.awaitTermination() // Wait for the computation to terminate
  }
}
```

Transformations on Dstreams-
UpdateStateByKey Operation

The master requires 2 threads to prevent from a starvation scenario.

- Create a local StreamingContext with 8 working threads (e.g., 2 cores) and batch interval of 5 seconds. local[*] : Run Spark locally with as many working threads as logical cores on your machine.

```
import org.apache.spark._
import org.apache.spark.streaming._

object StatefulNetworkWordCount {
  def updateFunction(newValues: Seq[Int], runningCount: Option[Int]): Option[Int] = {
    val newCount = runningCount.getOrElse(0) + newValues.sum
    Some(newCount)
  }

  def main(args: Array[String]) {
    val conf = new SparkConf().setMaster("local[8]").setAppName("StatefulNetworkWordCount")
    val ssc = new StreamingContext(conf, Seconds(5))
    ssc.checkpoint(".")
    val lines = ssc.socketTextStream("localhost", 9999)
    val words = lines.flatMap(_.split(" "))
    val pairs = words.map(word => (word, 1))
    val runningCounts = pairs.updateStateByKey[Int](updateFunction _)
    runningCounts.print()
    ssc.start() // Start the computation
    ssc.awaitTermination() // Wait for the computation to terminate
  }
}
```

Classification: Trusted

Dstreams-

- In Apache Spark, the master requires two threads because one thread will be used to run the receiver. The system cannot process the data if the number of allocated threads is less than 2.
- Starvation scenario in Spark Streaming refers to the problem when some tasks are not able to execute at all while others make progress.

The master requires 2 threads to prevent from a starvation scenario.

- Create a local StreamingContext with 8 working threads (e.g., 2 cores) and batch interval of 5 seconds. local[*] : Run Spark locally with as many working threads as logical cores on your machine.

```
, runningCount: Option[Int]): Option[Int] = {
e(0) + newValues.sum
```

```
def main(args: Array[String]) {
  val conf = new SparkConf().setMaster("local[8]").setAppName("StatefulNetworkWordCount")
  val ssc = new StreamingContext(conf, Seconds(5))
  ssc.checkpoint(".")
  val lines = ssc.socketTextStream("localhost", 9999)
  val words = lines.flatMap(_.split(" "))
  val pairs = words.map(word => (word, 1))
  val runningCounts = pairs.updateStateByKey[Int](updateFunction _)
  runningCounts.print()
  ssc.start() // Start the computation
  ssc.awaitTermination() // Wait for the computation to terminate
}
}
```

Transformations on Dstreams-
UpdateStateByKey Operation

- Create a local StreamingContext with 8 working threads (e.g., 2 cores) and batch interval of 5 seconds. local[*] : Run Spark locally with as many working threads as logical cores on your machine.

```
import org.apache.spark._
import org.apache.spark.streaming._

object StatefulNetworkWordCount {
  def updateFunction(newValues: Seq[Int], runningCount: Option[Int]): Option[Int] = {
    val newCount = runningCount.getOrElse(0) + newValues.sum
    Some(newCount)
  }

  def main(args: Array[String]) {
    val conf = new SparkConf().setMaster("local[8]").setAppName("StatefulNetworkWordCount")
    val ssc = new StreamingContext(conf, Seconds(5))
    ssc.checkpoint(".")
    val lines = ssc.socketTextStream("localhost", 9999)
    val words = lines.flatMap(_.split(" "))
    val pairs = words.map(word => (word, 1))
    val runningCounts = pairs.updateStateByKey[Int](updateFunction _)
    runningCounts.print()
    ssc.start() // Start the computation
    ssc.awaitTermination() // Wait for the computation to terminate
  }
}
```

Set checkpoint directory

Transformations on Dstreams-
UpdateStateByKey Operation

- Create a local StreamingContext with 8 working threads (e.g., 2 cores) and batch interval of 5 seconds. local[*] : Run Spark locally with as many working threads as logical cores on your machine.

```
import org.apache.spark._
import org.apache.spark.streaming._

object StatefulNetworkWordCount {
  def updateFunction(newValues: Seq[Int], runningCount: Option[Int]): Option[Int] = {
    val newCount = runningCount.getOrElse(0) + newValues.sum
    Some(newCount)
  }

  def main(args: Array[String]) {
    val conf = new SparkConf().setMaster("local[8]").setAppName("StatefulNetworkWordCount")
    val ssc = new StreamingContext(conf, Seconds(5))
    ssc.checkpoint(".")
    val lines = ssc.socketTextStream("localhost", 9999)
    val words = lines.flatMap(_.split(" "))
    val pairs = words.map(word => (word, 1))
    val runningCounts = pairs.updateStateByKey[Int](updateFunction _)
    runningCounts.print()
    ssc.start() // Start the computation
    ssc.awaitTermination() // Wait for the computation to terminate
  }
}
```

Create a DStream that will connect to hostname:port, like localhost:9999

Transformations on Dstreams-
UpdateStateByKey Operation

- Create a local StreamingContext with 8 working threads (e.g., 2 cores) and batch interval of 5 seconds. local[*] : Run Spark locally with as many working threads as logical cores on your machine.

```
import org.apache.spark._
import org.apache.spark.streaming._

object StatefulNetworkWordCount {
  def updateFunction(newValues: Seq[Int], runningCount: Option[Int]): Option[Int] = {
    val newCount = runningCount.getOrElse(0) + newValues.sum
    Some(newCount)
  }

  def main(args: Array[String]) {
    val conf = new SparkConf().setMaster("local[8]").setAppName("StatefulNetworkWordCount")
    val ssc = new StreamingContext(conf, Seconds(5))
    ssc.checkpoint(".")
    val lines = ssc.socketTextStream("localhost", 9999)
    val words = lines.flatMap(_.split(" "))
    val pairs = words.map(word => (word, 1))
    val runningCounts = pairs.updateStateByKey[Int](upda
    runningCounts.print()
    ssc.start() // Start the computation
    ssc.awaitTermination() // Wait for the computation to terminate
  }
}
```

Split lines (the entire file) into words and map each word to (word, 1) pairs.

Transformations on Dstreams-
UpdateStateByKey Operation

```
import org.apache.spark._
import org.apache.spark.streaming._

object StatefulNetworkWordCount {
  def updateFunction(newValues: Seq[Int], runningCount: Option[Int]): Option[Int] = {
    val newCount = runningCount.getOrElse(0) + newValues.sum
    Some(newCount)
  }

  def main(args: Array[String]) {
    val conf = new SparkConf().setMaster("local[8]").setAppName("StatefulNetworkWordCount")
    val ssc = new StreamingContext(conf, Seconds(5))
    ssc.checkpoint(".")
    val lines = ssc.socketTextStream("localhost", 9999)
    val words = lines.flatMap(_.split(" "))
    val pairs = words.map(word => (word, 1))
    val runningCounts = pairs.updateStateByKey[Int](updateFunction _)
    runningCounts.print()
    ssc.start() // Start the computation
    ssc.awaitTermination() // Wait for the computation to terminate
  }
}
```

Update state using the defined updateStateByKey

Transformations on Dstreams-
UpdateStateByKey Operation

some(expr) – a spark build-in function
Returns true if at least one value of expr is true

```
import org.apache.spark._
import org.apache.spark.streaming._

object StatefulNetworkWordCount {
  def updateFunction(newValues: Seq[Int], runningCount: Option[Int]): Option[Int] = {
    val newCount = runningCount.getOrElse(0) + newValues.sum
    Some(newCount)
  }

  def main(args: Array[String]) {
    val conf = new SparkConf().setMaster("local[8]").setAppName("StatefulNetworkWordCount")
    val ssc = new StreamingContext(conf, Seconds(5))
    ssc.checkpoint(".")
    val lines = ssc.socketTextStream("localhost", 9999)
    val words = lines.flatMap(_.split(" "))
    val pairs = words.map(word => (word, 1))
    val runningCounts = pairs.updateStateByKey[Int](updateFunction _)
    runningCounts.print()
    ssc.start() // Start the computation
    ssc.awaitTermination() // Wait for the computation to terminate
  }
}
```

Update state using the defined updateStateByKey

You can run this code based on what you learned in practical class
Tips: spark-submit --class org.example.statefulNetworkWordCount --master yarn --deploy-mode client statefulNetworkWordCount.jar

```
val runningCounts = pairs.updateStateByKey[Int](updateFunction )

def updateFunction
        (newValues: Seq[Int], runningCount: Option[Int]): Option[Int] = {
            // add the new values with the previous running count to get the new count
            val newCount = ...
            Some(newCount)
}
```

Q2 - What are information represented by newValues and runningCount respectively?

- newValues having a sequence of values for each key in new RDD
- newValue will be the latest value for each key of stream data processing so far.
- runningCount is the aggregation of values over the current stream RDD batch.
- runningCount having the count for each key of the previous RDDs

```
val runningCounts = pairs.updateStateByKey[Int](updateFunction )

def updateFunction
        (newValues: Seq[Int], runningCount: Option[Int]): Option[Int] = {
            // add the new values with the previous running count to get the new count
            val newCount = ...
            Some(newCount)
}
```

Q2 - What are information represented by newValues and runningCount respectively?

- ✓ newValues having a sequence of values for each key in new RDD
- ☐ newValue will be the latest value for each key of stream data processing so far.
- ☐ runningCount is the aggregation of values over the current stream RDD batch.
- ☐ runningCount having the count for each key of the previous RDDs

```
val runningCounts = pairs.updateStateByKey[Int](updateFunction )

def updateFunction
        (newValues: Seq[Int], runningCount: Option[Int]): Option[Int] = {
            // add the new values with the previous running count to get the new count
            val newCount = ...
            Some(newCount)
}
```

Q2 - What are information represented by newValues and runningCount respectively?

- ✓ newValues having a sequence of values for each key in new RDD
- ✗ newValue will be the latest value for each key of stream data processing so far.
- ❑ runningCount is the aggregation of values over the current stream RDD batch.
- ❑ runningCount having the count for each key of the previous RDDs

```
val runningCounts = pairs.updateStateByKey[Int](updateFunction )

def updateFunction
        (newValues: Seq[Int], runningCount: Option[Int]): Option[Int] = {
            // add the new values with the previous running count to get the new count
            val newCount = ...
            Some(newCount)
}
```

Q2 - What are information represented by newValues and runningCount respectively?

- ✓ newValues having a sequence of values for each key in new RDD
- ✗ newValue will be the latest value for each key of stream data processing so far.
- ✗ runningCount is the aggregation of values over the current stream RDD batch.
- ❑ runningCount having the count for each key of the previous RDDs

```
val runningCounts = pairs.updateStateByKey[Int](updateFunction )

def updateFunction
        (newValues: Seq[Int], runningCount: Option[Int]): Option[Int] = {
            // add the new values with the previous running count to get the new count
            val newCount = ...
            Some(newCount)
}
```

Q2 - What are information represented by newValues and runningCount respectively?

- ✓ newValues having a sequence of values for each key in new RDD
- ✗ newValue will be the latest value for each key of stream data processing so far.
- ✗ runningCount is the aggregation of values over the current stream RDD batch.
- ✓ runningCount having the count for each key of the previous RDDs

Q3 - Regarding the following operation, which statement is incorrect?

```
val windowedWordCounts =
pairs.reduceByKeyAndWindow((a:Int,b:Int)=> (a + b),
Seconds(100), Seconds(10), 3)
```

- The slide interval is 3 seconds
- In local mode, the window size is 2 by default
- The reduce function over batches in a sliding window is applied
- In cluster mode, the window size is determined by the config property spark.default.parallelism.

DStream Operations

Transformation

- Map
- flatMap
- Fliter
- Repartition
- Union
- Count
- Reduce
- countByValue
- reduceByKey
- Join
- Cogroup
- Transform

Stateful Transformation

- updateStateByKey

maintains per-key state across batches until explicitly removed.

- Window Operations

retains data for a bounded time window

  - Window
  - countByWindow
  - reduceByWindow
  - reduceByKeyAndWindow

Transformations - Window Operations

Spark Streaming also provides *windowed computations*, which allow you to apply transformations over a sliding window of data.

Stateful Transformation

- updateStateByKey

maintains per-key state across batches until explicitly removed.

- Window Operations

retains data for a bounded time window

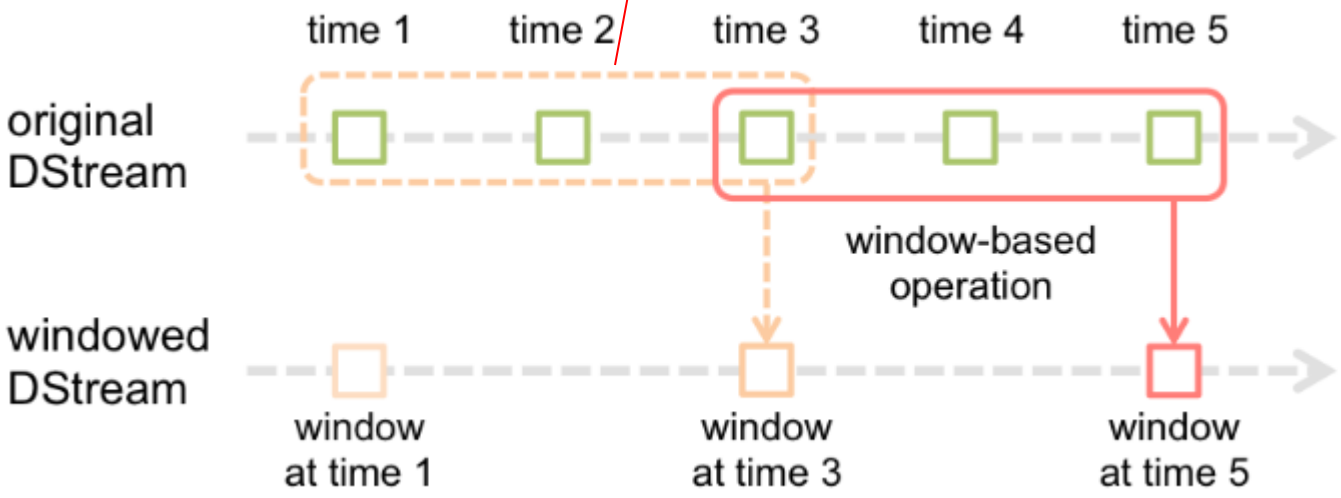
  - Window
  - countByWindow
  - reduceByWindow
  - reduceByKeyAndWindow

Transformations - Window Operations

Spark Streaming also provides *windowed computations*, which allow you to apply transformations over a sliding window of data.

a sliding window

The following figure illustrates this sliding window.

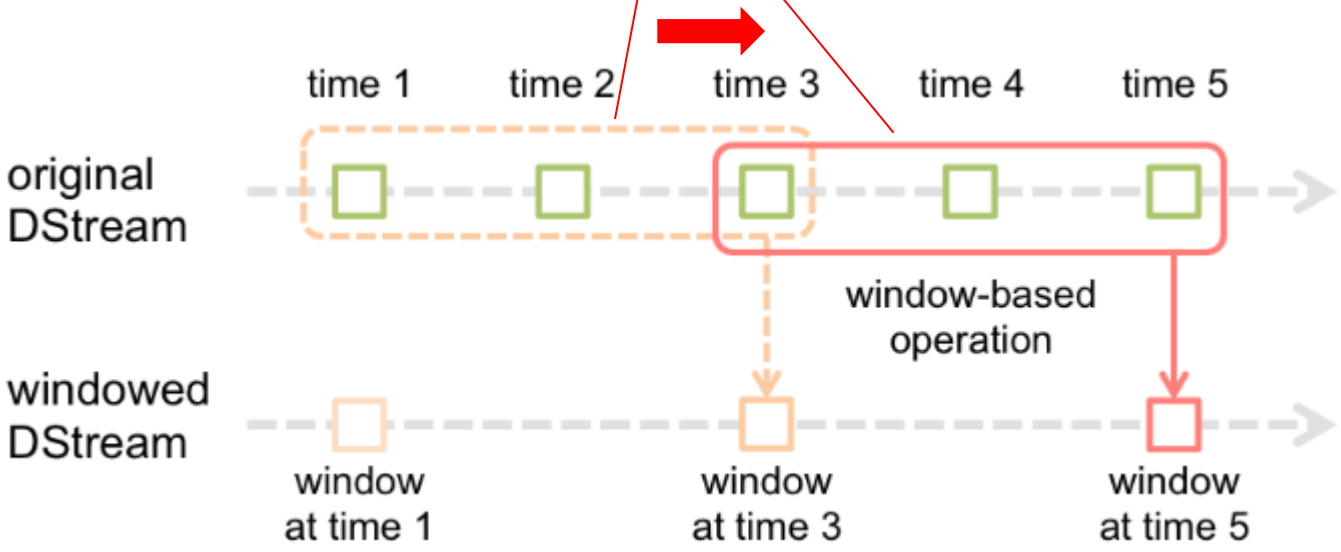


Transformations - Window Operations

Spark Streaming also provides *windowed computations*, which allow you to apply transformations over a sliding window of data.

a sliding window

The following figure illustrates this sliding window.



Transformations - Window Operations

reduceByKeyAndWindow operation

reduceByKeyAndWindow

(*func*, *windowLength*, *slideInterval*, [*numTasks*])

Stateful Transformation

- updateStateByKey

maintains per-key state across batches until explicitly removed.

- Window Operations

retains data for a bounded time window

  - Window
  - countByWindow
  - reduceByWindow
  - reduceByKeyAndWindow

Transformations - Window Operations

reduceByKeyAndWindow operation

reduceByKeyAndWindow

(*func*, *windowLength*, *slideInterval*, [*numTasks*])

When called on a DStream of (K, V) pairs, it returns a new DStream of (K, V) pairs where the values for each key are aggregated using the given reduce function func over batches in a sliding window.

Transformations - Window Operations

reduceByKeyAndWindow operation

reduceByKeyAndWindow

(*func*, *windowLength*, *slideInterval*, [*numTasks*])

When called on a DStream of (K, V) pairs, it returns a new DStream of (K, V) pairs where the values for each key are aggregated using the given reduce function func over batches in a sliding window.

Transformations - Window Operations

reduceByKeyAndWindow operation

reduceByKeyAndWindow

(*func*, *windowLength*, *slideInterval*, [*numTasks*])

When called on a DStream of (K, V) pairs, it returns a new DStream of (K, V) pairs where the values for each key are aggregated using the given reduce function func over batches in a sliding window.

- By default, this uses Spark's default number of parallel tasks
  - In local mode, the number is 2
  - In cluster mode, the number is determined by the config property spark.default.parallelism
- You can pass an optional numTasks argument to set a different number of tasks.

Transformations - Window Operations

reduceByKeyAndWindow operation

reduceByKeyAndWindow

(*func*, *windowLength*, *slideInterval*, [*numTasks*])

Spark's parallelism unit is partitions

When called on a DStream of (K, V) pairs, it returns a new DStream of (K, V) pairs where the values for each key are aggregated using the given reduce function func over batches in a sliding window.

- By default, this uses Spark's default number of parallel tasks
  - In local mode, the number is 2
  - In cluster mode, the number is determined by the config property spark.default.parallelism
- You can pass an optional numTasks argument to set a different number of tasks.

RMIT Classification: Trusted

Transformations - Window Operations

reduceByKeyAndWindow operation

reduceByKeyAndWindow

(*func*, *windowLength*, *slideInterval*, [*numTasks*])

When called on a DStream of (K, V) pairs, it returns a new DStream of (K, V) pairs where the values for each key are aggregated using the given reduce function func over batches in a sliding window.

- By default, this uses Spark's default number of parallel tasks
  - In local mode, the number is 2
  - In cluster mode, the number is determined by the config property spark.default.parallelism
- You can pass an optional numTasks argument to set a different number of tasks.

Transformations - Window Operations

Example: Generating word counts over the last 30 seconds of data, every 20 seconds.

Transformations - Window Operations

Example: Generating word counts over the last 30 seconds of data, every 20 seconds.

- Apply the reduceByKey operation on the key-value pairs in DStream over the last 30 seconds of data, and repeat for every 20 seconds.

Transformations - Window Operations

Example: Generating word counts over the last 30 seconds of data, every 20 seconds.

- Apply the reduceByKey operation on the key-value pairs in DStream over the last 30 seconds of data, and repeat for every 20 seconds. To do this, we can simply use reduceByKeyAndWindow.

Transformations - Window Operations

Example: Generating word counts over the last 30 seconds of data, every 20 seconds.

- Apply the reduceByKey operation on the key-value pairs in DStream over the last 30 seconds of data, and repeat for every 20 seconds. To do this, we can simply use reduceByKeyAndWindow.

```
// Reduce last 30 seconds of data, every 20 seconds
val windowedWordCounts = pairs.reduceByKeyAndWindow
                ((a:Int,b:Int)=> (a + b), Seconds(30), Seconds(20))
```

reduceByKeyAndWindow (*func*, *windowLength*, *slideInterval*, [*numTasks*])

Transformations - Window Operations

Example: Generating word counts over the last 30 seconds of data, every 20 seconds.

- Apply the reduceByKey operation on the key-value pairs in DStream over the last 30 seconds of data, and repeat for every 20 seconds. To do this, we can simply use reduceByKeyAndWindow.

```
// Reduce last 30 seconds of data, every 20 seconds
val windowedWordCounts = pairs.reduceByKeyAndWindow
                ((a:Int,b:Int)=> (a + b), Seconds(30), Seconds(20))
```

reduceByKeyAndWindow (*func*, *windowLength*, *slideInterval*, [*numTasks*])

Transformations - Window Operations

Example: Generating word counts over the last 30 seconds of data, every 20 seconds.

- Apply the reduceByKey operation on the key-value pairs in DStream over the last 30 seconds of data, and repeat for every 20 seconds. To do this, we can simply use reduceByKeyAndWindow.

```
// Reduce last 30 seconds of data, every 20 seconds
val windowedWordCounts = pairs.reduceByKeyAndWindow
                ((a:Int,b:Int)=> (a + b), Seconds(30), Seconds(20))
```

reduceByKeyAndWindow (*func*, *windowLength*, *slideInterval*, [*numTasks*])

Transformations - Window Operations

Example: Generating word counts over the last 30 seconds of data, every 20 seconds.

- Apply the reduceByKey operation on the key-value pairs in DStream over the last 30 seconds of data, and repeat for every 20 seconds. To do this, we can simply use reduceByKeyAndWindow.

```
// Reduce last 30 seconds of data, every 20 seconds
val windowedWordCounts = pairs.reduceByKeyAndWindow
                ((a:Int,b:Int)=> (a + b), Seconds(30), Seconds(20))
```

lambda function

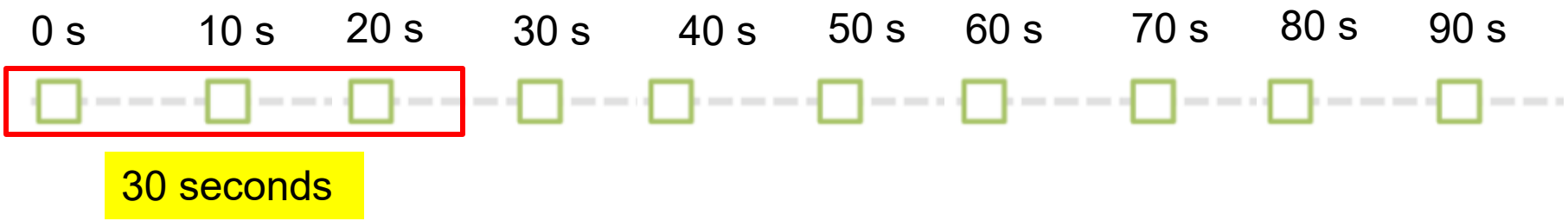
reduceByKeyAndWindow (*func*, *windowLength*, *slideInterval*, [*numTasks*])

Transformations - Window Operations

Example: Generating word counts over the last 30 seconds of data, every 20 seconds.

- Apply the reduceByKey operation on the key-value pairs in DStream over the last 30 seconds of data, and repeat for every 20 seconds. To do this, we can simply use reduceByKeyAndWindow.

```
// Reduce last 30 seconds of data, every 20 seconds
val windowedWordCounts = pairs.reduceByKeyAndWindow
                ((a:Int,b:Int)=> (a + b), Seconds(30), Seconds(20))
```

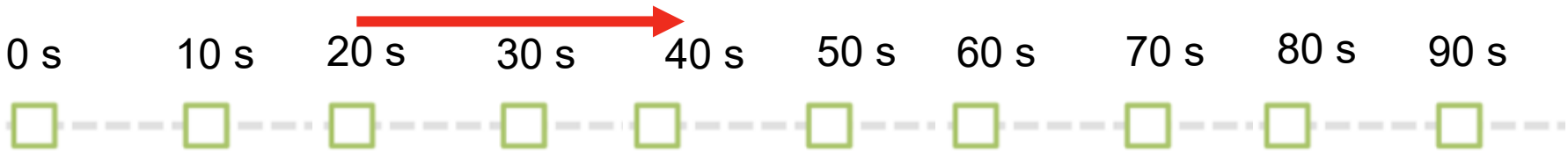


Transformations - Window Operations

Example: Generating word counts over the last 30 seconds of data, every 20 seconds.

- Apply the reduceByKey operation on the key-value pairs in DStream over the last 30 seconds of data, and repeat for every 20 seconds. To do this, we can simply use reduceByKeyAndWindow.

```
// Reduce last 30 seconds of data, every 20 seconds
val windowedWordCounts = pairs.reduceByKeyAndWindow
                ((a:Int,b:Int)=> (a + b), Seconds(30), Seconds(20))
```

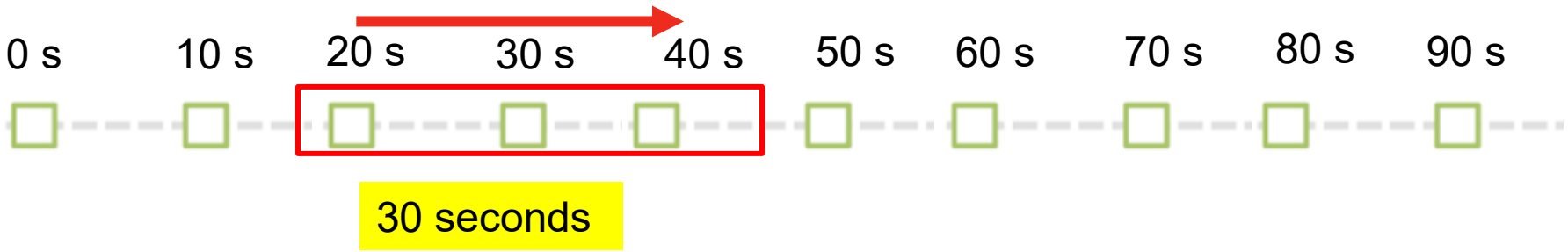


Transformations - Window Operations

Example: Generating word counts over the last 30 seconds of data, every 20 seconds.

- Apply the reduceByKey operation on the key-value pairs in DStream over the last 30 seconds of data, and repeat for every 20 seconds. To do this, we can simply use reduceByKeyAndWindow.

```
// Reduce last 30 seconds of data, every 20 seconds
val windowedWordCounts = pairs.reduceByKeyAndWindow
                ((a:Int,b:Int)=> (a + b), Seconds(30), Seconds(20))
```



Transformations - Window Operations

Transformation	Meaning
window(*windowLength*, *slideInterval*)	Return a new DStream which is computed based on windowed batches of the source DStream.
countByWindow(*windowLength*, *slideInterval*)	Return a sliding wir
reduceByWindow(*func*, *windowLength*, *slideInterval*)	Return a new singl elements in the stre function should be be computed corre

Stateful Transformation

- updateStateByKey

maintains per-key state across batches until explicitly removed.

- Window Operations

retains data for a bounded time window

- Window
- countByWindow
- reduceByWindow
- reduceByKeyAndWindow

Transformations - Window Operations

reduceByKeyAndWindow(*func*, *windowLength*, *slideInterval*, [*numTasks*])	When called on a DStream of (K, V) pairs, returns a new DStream of (K, V) pairs where the values for each key are aggregated using the given reduce function *func* over batches in a sliding window. **Note:** By default, this uses Spark's default number of parallel tasks (2 for local mode, and in cluster mode the number is determined by the config property `spark.default.parallel` `numTasks` argument to set a
countByValueAndWindow(*windowLength*, *slideInterval*, [*numTasks*])	When called on a DStream Long) pairs where the value window. Like in `reduceByKe` configurable through an opt

Stateful Transformation

- updateStateByKey

maintains per-key state across batches until explicitly removed.

- Window Operations

retains data for a bounded time window
  - Window
  - countByWindow
  - reduceByWindow
  - reduceByKeyAndWindow

Q3 - Regarding the following operation, which statement is incorrect?

```
val windowedWordCounts =
pairs.reduceByKeyAndWindow((a:Int,b:Int)=> (a + b),
Seconds(100), Seconds(10), 3)
```

- The slide interval is 3 seconds
- In local mode, the window size is 2 by default
- The reduce function over batches in a sliding window is applied
- In cluster mode, the window size is determined by the config property spark.default.parallelism.

Q3 - Regarding the following operation, which statement is incorrect?

```
val windowedWordCounts =
pairs.reduceByKeyAndWindow((a:Int,b:Int)=> (a + b),
Seconds(100), Seconds(10), 3)
```



- The slide interval is 3 seconds
- ☐ In local mode, the window size is 2 by default
- ☐ The reduce function over batches in a sliding window is applied
- ☐ In cluster mode, the window size is determined by the config property spark.default.parallelism.

Q3 - Regarding the following operation, which statement is incorrect?

```
val windowedWordCounts =
pairs.reduceByKeyAndWindow((a:Int,b:Int)=> (a + b),
Seconds(100), Seconds(10), 3)
```

- ✗ The slide interval is 3 seconds
- ✗ In local mode, the window size is 2 by default
- ☐ The reduce function over batches in a sliding window is applied
- ☐ In cluster mode, the window size is determined by the config property spark.default.parallelism.

Q3 - Regarding the following operation, which statement is incorrect?

```
val windowedWordCounts =
pairs.reduceByKeyAndWindow((a:Int,b:Int)=> (a + b),
Seconds(100), Seconds(10), 3)
```

- ✗ The slide interval is 3 seconds
- ✗ In local mode, the window size is 2 by default
- ✓ The reduce function over batches in a sliding window is applied
- ❑ In cluster mode, the window size is determined by the config property spark.default.parallelism.

Q3 - Regarding the following operation, which statement is incorrect?

```
val windowedWordCounts =
pairs.reduceByKeyAndWindow((a:Int,b:Int)=> (a + b),
Seconds(100), Seconds(10), 3)
```

- ✗ The slide interval is 3 seconds
- ✗ In local mode, the window size is 2 by default
- ✓ The reduce function over batches in a sliding window is applied
- ✗ In cluster mode, the window size is determined by the config property spark.default.parallelism.

Q4 - Which of the following statements is/are correct?

```
A:
dstream.foreachRDD { rdd =>
  rdd.foreach { record =>
    val connection = createNewConnection()
    connection.send(record)
    connection.close()
  }
}
```

```
B:
dstream.foreachRDD { rdd =>
  rdd.foreachPartition { partitionOfRecords =>
    val connection = createNewConnection()
    partitionOfRecords.foreach(record => connec
tion.send(record))
    connection.close()
  }
}
```

- B is generally a better implementation compared with A.
- A - create a new connection for every record.
- Typically, creating a connection object has time and resource overheads. Therefore, creating and destroying a connection object for each record can incur unnecessarily high overheads and can significantly reduce the overall throughput of the system.
- B - create a single connection object and send all the records in a RDD partition

DStream Operations

Transformation

- Map
- flatMap
- Fliter
- Repartition
- Union
- Count
- Reduce
- countByValue
- reduceByKey
- Join
- Cogroup
- Transform

Stateful Transformation

- updateStateByKey

maintains per-key state across batches until explicitly removed.

- Window Operations

retains data for a bounded time window

  - Window
  - countByWindow
  - reduceByWindow
  - reduceByKeyAndWindow

Action

- Output Operations
  - print()
  - saveAsTextFiles()
  - saveAsObjectFiles()
  - saveAsHadoopFiles()
  - foreachRDD(func)

Action - Output Operations

- Output operations allow DStream's data to be pushed out to external systems like a database or a file system.
- Since the output operations actually allow the transformed data to be consumed by external systems, they trigger the actual execution of all the DStream transformations

Action - Output Operations

- Output operations allow DStream's data to be pushed out to external systems like a database or a file system.
- Since the output operations actually allow the transformed data to be consumed by external systems, they trigger the actual execution of all the DStream transformations.

Action - Output Operations

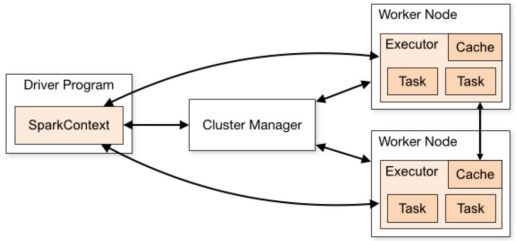
- Output operations allow DStream's data to be pushed out to external systems like a database or a file system.
- Since the output operations actually allow the transformed data to be consumed by external systems, they trigger the actual execution of all the DStream transformations -- similar to actions for RDDs.

Action - Output Operations

- Design Patterns for using foreachRDD

Writing data to external system requires creating a connection object (e.g. TCP connection to a remote server) and using it to send data to a remote system.

Action - Output Operations



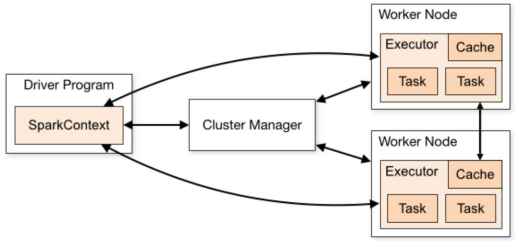
- Design Patterns for using foreachRDD

Writing data to external system requires creating a connection object (e.g. TCP connection to a remote server) and using it to send data to a remote system.

- For this purpose, a developer may try creating a connection object at the Spark driver, and then try to use it in a Spark worker to save records in the RDDs

```
dstream.foreachRDD { rdd =>
  val connection = createNewConnection()  // executed at the driver
  rdd.foreach { record =>
    connection.send(record) // executed at the worker
  }
}
```

Action - Output Operations



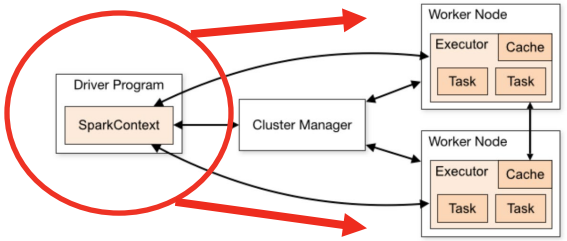
- Design Patterns for using foreachRDD

Writing data to external system requires creating a connection object (e.g. TCP connection to a remote server) and using it to send data to a remote system.

- For this purpose, a developer may try creating a connection object at the Spark driver, and then try to use it in a Spark worker to save records in the RDDs

```
dstream.foreachRDD { rdd =>
  val connection = createNewConnection()  // executed at the driver
  rdd.foreach { record =>
    connection.send(record) // executed at the worker
  }
}
```

Action - Output Operations



Design Patterns for using foreachRDD

Writing data to external system requires creating a connection object (e.g. TCP connection to a remote server) and using it to send data to a remote system.

- For this purpose, a developer may try creating a connection object at the Spark driver, and then try to use it in a Spark worker to save records in the RDDs

```
dstream.foreachRDD { rdd =>
  val connection = createNewCon...ion()   // executed at the driver
  rdd.foreach { record =>
    connection.send(record)  // executed at the worker
  }
}
```

This is incorrect because this requires the connection object to be serialized and sent from the driver to the worker.

Action - Output Operations

The correct solution is to create the connection object at the worker.

Action - Output Operations

The correct solution is to create the connection object at the worker.

- This can lead to another common mistake - creating a new connection for every record.

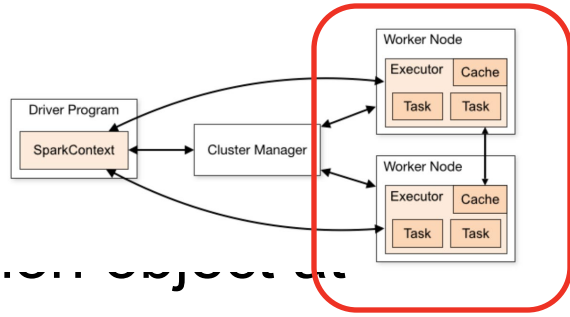
```
dstream.foreachRDD { rdd =>
  rdd.foreach { record =>
    val connection = createNewConnection()
    connection.send(record)
    connection.close()
  }
}
```

```
dstream.foreachRDD { rdd =>
  val connection = createNewConnection()  // executed at the driver
  rdd.foreach { record =>
    connection.send(record) // executed at the worker
  }
}
```

- This can lead to another common mistake - creating a new connection for every record.

```
dstream.foreachRDD { rdd =>
  rdd.foreach { record =>
    val connection = createNewConnection()
    connection.send(record)
    connection.close()
  }
}
```

Action - Output Operations



The correct solution is to create the connect… object at the worker.

- This can lead to another common mistake - creating a new connection for every record.

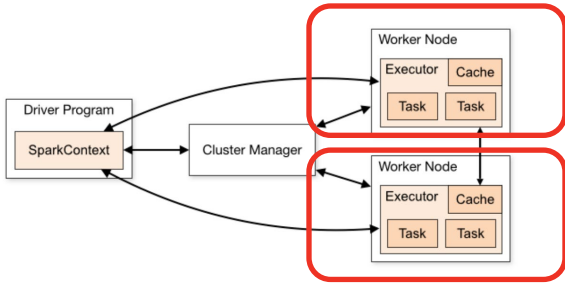
```
dstream.foreachRDD { rdd =>
  rdd.foreach { record =>
    val connection = createNewConnection()
    connection.send(record)
    connection.close()
  }
}
```

Typically, creating a connection object has time and resource overheads. Therefore, creating and destroying a connection object for each record can incur unnecessarily high overheads and can significantly reduce the overall throughput of the system.

Action - Output Operations

- A better solution is to use rdd.foreachPartition - create a single connection object and send all the records in a RDD partition using that connection.

Action - Output Operations



- A better solution is to use rdd.foreachPartition - create a single connection object and send all the records in a RDD partition using that connection.

```
dstream.foreachRDD { rdd =>
  rdd.foreachPartition { partitionOfRecords =>
    val connection = createNewConnection()
    partitionOfRecords.foreach(record => connection.send(record))
    connection.close()
  }
}
```

This amortizes the connection creation overheads over many records.

```
dstream.foreachRDD { rdd =>
  rdd.foreach { record =>
    val connection = createNewConnection()
    connection.send(record)
    connection.close()
  }
}
```

reachPartition - create a
nd all the records in a
RDD partition using that connection.

```
dstream.foreachRDD { rdd =>
  rdd.foreachPartition { partitionOfRecords =>
    val connection = createNewConnection()
    partitionOfRecords.foreach(record => connection.send(record))
    connection.close()
  }
}
```

This amortizes the connection creation overheads over many records.

Action - Output Operations

Finally, this can be further optimized by reusing connection objects across multiple RDDs/batches.

Action - Output Operations

Finally, this can be further optimized by reusing connection objects across multiple RDDs/batches.

- One can maintain a static pool of connection objects that can be reused as RDDs of multiple batches are pushed to the external system, thus further reducing the overheads.

Action - Output Operations

Finally, this can be further optimized by reusing connection objects across multiple RDDs/batches.

- One can maintain a static pool of connection objects that can be reused as RDDs of multiple batches are pushed to the external system, thus further reducing the overheads.

```
dstream.foreachRDD { rdd =>
  rdd.foreachPartition { partitionOfRecords =>
    // ConnectionPool is a static, lazily initialized pool of connections
    val connection = ConnectionPool.getConnection()
    partitionOfRecords.foreach(record => connection.send(record))
    ConnectionPool.returnConnection(connection)  // return to the pool for future reuse
  }
}
```

Action - Output Operations

```
dstream.foreachRDD { rdd =>
  rdd.foreachPartition { partitionOfRecords =>
    val connection = createNewConnection()
    partitionOfRecords.foreach(record => connection.send(record))
    connection.close()
  }
}
```

```
dstream.foreachRDD { rdd =>
  rdd.foreachPartition { partitionOfRecords =>
    // ConnectionPool is a static, lazily initialized pool of connections
    val connection = ConnectionPool.getConnection()
    partitionOfRecords.foreach(record => connection.send(record))
    ConnectionPool.returnConnection(connection)  // return to the pool for future reuse
  }
}
```

Caching / Persistence

Similar to RDDs, DStreams also allow developers to persist the stream's data in memory.

Caching / Persistence

Similar to RDDs, DStreams also allow developers to persist the stream's data in memory.

- Using the persist() method on a DStream will automatically persist every RDD of that DStream in memory.

Caching / Persistence

Similar to RDDs, DStreams also allow developers to persist the stream's data in memory.

- Using the persist() method on a DStream will automatically persist every RDD of that DStream in memory.

This is useful if the data in the DStream will be computed multiple times (e.g., multiple operations on the same data).

C ce

Stateful Transformation

- updateStateByKey

maintains per-key state across batches until explicitly removed.c

- Window Operations

retains data for a bounded time window
  - Window
  - countByWindow
  - reduceByWindow
  - reduceByKeyAndWindow

S also allow developers to persist th .

- a DStream will automatically persist memory.

DStream will be computed multiple on the same data).

DStreams generated are automatically persisted in memory, without the developer calling persist().

- window-based operations
  - like reduceByWindow and reduceByKeyAndWindow
- state-based operations
  - like updateStateByKey,

Q4 - Which of the following statements is/are correct?

```
A:
dstream.foreachRDD { rdd =>
  rdd.foreach { record =>
    val connection = createNewConnection()
    connection.send(record)
    connection.close()
  }
}
```

```
B:
dstream.foreachRDD { rdd =>
  rdd.foreachPartition { partitionOfRecords =>
    val connection = createNewConnection()
    partitionOfRecords.foreach(record => connec
tion.send(record))
    connection.close()
  }
}
```

- B is generally a better implementation compared with A.
- A - create a new connection for every record.
- Typically, creating a connection object has time and resource overheads. Therefore, creating and destroying a connection object for each record can incur unnecessarily high overheads and can significantly reduce the overall throughput of the system.
- B - create a single connection object and send all the records in a RDD partition

Q4 - Which of the following statements is/are correct?

```
A:
dstream.foreachRDD { rdd =>
  rdd.foreach { record =>
    val connection = createNewConnection()
    connection.send(record)
    connection.close()
  }
}
```

```
B:
dstream.foreachRDD { rdd =>
  rdd.foreachPartition { partitionOfRecords =>
    val connection = createNewConnection()
    partitionOfRecords.foreach(record => connec
tion.send(record))
    connection.close()
  }
}
```



B is generally a better implementation compared with A.

- A - create a new connection for every record.
- Typically, creating a connection object has time and resource overheads. Therefore, creating and destroying a connection object for each record can incur unnecessarily high overheads and can significantly reduce the overall throughput of the system.
- B - create a single connection object and send all the records in a RDD partition

Q4 - Which of the following statements is/are correct?

```
A:
dstream.foreachRDD { rdd =>
  rdd.foreach { record =>
    val connection = createNewConnection()
    connection.send(record)
    connection.close()
  }
}
```

```
B:
dstream.foreachRDD { rdd =>
  rdd.foreachPartition { partitionOfRecords =>
    val connection = createNewConnection()
    partitionOfRecords.foreach(record => connec
tion.send(record))
    connection.close()
  }
}
```

- ✓ B is generally a better implementation compared with A.
- ✓ A - create a new connection for every record.
  - Typically, creating a connection object has time and resource overheads. Therefore, creating and destroying a connection object for each record can incur unnecessarily high overheads and can significantly reduce the overall throughput of the system.
  - B - create a single connection object and send all the records in a RDD partition

Q4 - Which of the following statements is/are correct?

```
A:
dstream.foreachRDD { rdd =>
  rdd.foreach { record =>
    val connection = createNewConnection()
    connection.send(record)
    connection.close()
  }
}
```

```
B:
dstream.foreachRDD { rdd =>
  rdd.foreachPartition { partitionOfRecords =>
    val connection = createNewConnection()
    partitionOfRecords.foreach(record => connec
tion.send(record))
    connection.close()
  }
}
```

- ✓ B is generally a better implementation compared with A.
- ✓ A - create a new connection for every record.
- ✓ Typically, creating a connection object has time and resource overheads. Therefore, creating and destroying a connection object for each record can incur unnecessarily high overheads and can significantly reduce the overall throughput of the system.
- B - create a single connection object and send all the records in a RDD partition

Q4 - Which of the following statements is/are correct?

```
A:
dstream.foreachRDD { rdd =>
  rdd.foreach { record =>
    val connection = createNewConnection()
    connection.send(record)
    connection.close()
  }
}
```

```
B:
dstream.foreachRDD { rdd =>
  rdd.foreachPartition { partitionOfRecords =>
    val connection = createNewConnection()
    partitionOfRecords.foreach(record => connec
tion.send(record))
    connection.close()
  }
}
```

- B is generally a better implementation compared with A.
- A - create a new connection for every record.
- Typically, creating a connection object has time and resource overheads. Therefore, creating and destroying a connection object for each record can incur unnecessarily high overheads and can significantly reduce the overall throughput of the system.
- B - create a single connection object and send all the records in a RDD partition

Q5 - What are the common characteristics of the following two types of operations?

>> Window-based operations

- like reduceByWindow and reduceByKeyAndWindow

>> State-based operations

- like updateStateByKey

- DStreams generated are automatically persisted in memory for both.
- They can be performed without explicitly persisted in memory.
- Both of them require checkpointing.
- window-based operations are more useful than state-based operations in Spark streaming.

Q5 - What are the common characteristics of the following two types of operations?

>> Window-based operations

- like reduceByWindow and reduceByKeyAndWindow

>> State-based operations

- like updateStateByKey

- DStreams generated are automatically persisted in memory for both.
- They can be performed without explicitly persisted in memory.
- Both of them require checkpointing.
- window-based operations are more useful than state-based operations in Spark streaming.

Q5 - What are the common characteristics of the following two types of operations?

>> Window-based operations

- like reduceByWindow and reduceByKeyAndWindow

>> State-based operations

- like updateStateByKey

- ✓ DStreams generated are automatically persisted in memory for both.
- ☐ They can be performed without explicitly persisted in memory.
- ☐ Both of them require checkpointing.
- ☐ window-based operations are more useful than state-based operations in Spark streaming.

Q5 - What are the common characteristics of the following two types of operations?

>> Window-based operations

- like reduceByWindow and reduceByKeyAndWindow

>> State-based operations

- like updateStateByKey

- ✓ DStreams generated are automatically persisted in memory for both.
- ✓ They can be performed without explicitly persisted in memory.
- ☐ Both of them require checkpointing.
- ☐ window-based operations are more useful than state-based operations in Spark streaming.

Q5 - What are the common characteristics of the following two types of operations?

>> Window-based operations

- like reduceByWindow and reduceByKeyAndWindow

>> State-based operations

- like updateStateByKey

- ✓ DStreams generated are automatically persisted in memory for both.
- ✓ They can be performed without explicitly persisted in memory.
- ✓ Both of them require checkpointing.
- ☐ window-based operations are more useful than state-based operations in Spark streaming.

Q5 - What are the common characteristics of the following two types of operations?

>> Window-based operations

- like reduceByWindow and reduceByKeyAndWindow

>> State-based operations

- like updateStateByKey



DStreams generated are automatically persisted in memory for both.



They can be performed without explicitly persisted in memory.



Both of them require checkpointing.



window-based operations are more useful than state-based operations in Spark streaming.

Q6 - Simple streaming applications without stateful transformations can be run without enabling checkpointing.

- True
- False

Checkpointing

A streaming application must operate 24/7 and hence must be resilient to failures unrelated to the application logic (e.g., system failures, JVM crashes, etc.).

Checkpointing

A streaming application must operate 24/7 and hence must be resilient to failures unrelated to the application logic (e.g., system failures, JVM crashes, etc.).

- For this to be possible, Spark Streaming needs to checkpoint enough information to a fault-tolerant storage system such that it can recover from failures.

Checkpointing

A streaming application must operate 24/7 and hence must be resilient to failures unrelated to the application logic (e.g., system failures, JVM crashes, etc.).

- For this to be possible, Spark Streaming needs to checkpoint enough information to a fault-tolerant storage system such that it can recover from failures.

Transformations on Dstreams-
UpdateStateByKey Operation

- Create a local StreamingContext with 8 working threads and batch interval of 5 seconds.

```
import org.apache.spark._
import org.apache.spark.streaming._

object StatefulNetworkWordCount {
  def updateFunction(newValues: Seq[Int], runningCount: Option[Int]): Option[Int] = {
    val newCount = runningCount.getOrElse(0) + newValues.sum
    Some(newCount)
  }

  def main(args: Array[String]) {
    val conf = new SparkConf().setMaster("local[8]").setAppName("StatefulNetworkWordCou
    val ssc = new StreamingContext(conf, Seconds(5))
    ssc.checkpoint(".")
    val lines = ssc.socketTextStream("localhost", 9999)
    val words = lines.flatMap(_.split(" "))
    val pairs = words.map(word => (word, 1))
    val runningCounts = pairs.updateStateByKey[Int](updateFunction _)
    runningCounts.print()
    ssc.start() // Start the computation
    ssc.awaitTermination() // Wait for the computation to terminate
  }
}
```

Set checkpoint directory

Checkpointing

A streaming application must operate 24/7 and hence must be resilient to failures unrelated to the application logic (e.g., system failures, JVM crashes, etc.).

- For this to be possible, Spark Streaming needs to checkpoint enough information to a fault-tolerant storage system such that it can recover from failures.
- There are two types of data that are checkpointed.
  - Metadata checkpointing
  - Data checkpointing

Checkpointing

Metadata checkpointing

- Saving of the information defining the streaming computation to fault-tolerant storage like HDFS. This is used to recover from failure of the node running the driver of the streaming.
- Metadata includes:
  - Configuration - The configuration that was used to create the streaming application.
  - DStream operations - The set of DStream operations that define the streaming application.
  - Incomplete batches - Batches whose jobs are queued but have not completed yet.

Checkpointing

Metadata checkpointing

- Saving of the information defining the streaming computation to fault-tolerant storage like HDFS. This is used to recover from failure of the node running the driver of the streaming.
- Metadata includes:
  - Configuration - The configuration that was used to create the streaming application.
  - DStream operations - The set of DStream operations that define the streaming application.
  - Incomplete batches - Batches whose jobs are queued but have not completed yet.

Checkpointing

Metadata checkpointing

- Saving of the information defining the streaming computation to fault-tolerant storage like HDFS. This is used to recover from failure of the node running the driver of the streaming.
- Metadata includes:
  - Configuration - The configuration that was used to create the streaming application.
  - DStream operations - The set of DStream operations that define the streaming application.
  - Incomplete batches - Batches whose jobs are queued but have not completed yet.

Checkpointing

Metadata checkpointing

- Saving of the information defining the streaming computation to fault-tolerant storage like HDFS. This is used to recover from failure of the node running the driver of the streaming.
- Metadata includes:
  - Configuration - The configuration that was used to create the streaming application.
  - DStream operations - The set of DStream operations that define the streaming application.
  - Incomplete batches - Batches whose jobs are queued but have not completed yet.

Checkpointing

Metadata checkpointing

- Saving of the information defining the streaming computation to fault-tolerant storage like HDFS. This is used to recover from failure of the node running the driver of the streaming.
- Metadata includes:
  - Configuration - The configuration that was used to create the streaming application.
  - DStream operations - The set of DStream operations that define the streaming application.
  - Incomplete batches - Batches whose jobs are queued but have not completed yet.

Checkpointing

Metadata checkpointing

- Saving of the information defining the streaming computation to fault-tolerant storage like HDFS. This is used to recover from failure of the node running the driver of the streaming.
- Metadata includes:
  - Configuration - The configuration that was used to create the streaming application.
  - DStream operations - The set of DStream operations that define the streaming application.
  - Incomplete batches - Batches whose jobs are queued but have not completed yet.

RMIT Classification: Trusted

Checkpointing

Data checkpointing

- Saving of the generated RDDs to reliable storage. This is necessary in some *stateful* transformations that combine data across multiple batches.
  - In such transformations, the generated RDDs depend on RDDs of previous batches, which causes the length of the dependency chain to keep increasing with time.

Checkpointing

Data checkpointing

- Saving of the generated RDDs to reliable storage. This is necessary in some *stateful* transformations that combine data across multiple batches.
  - In such transformations, the generated RDDs depend on RDDs of previous batches, which causes the length of the dependency chain to keep increasing with time.

When to enable Checkpointing

Checkpointing must be enabled for applications with any of the following requirements:

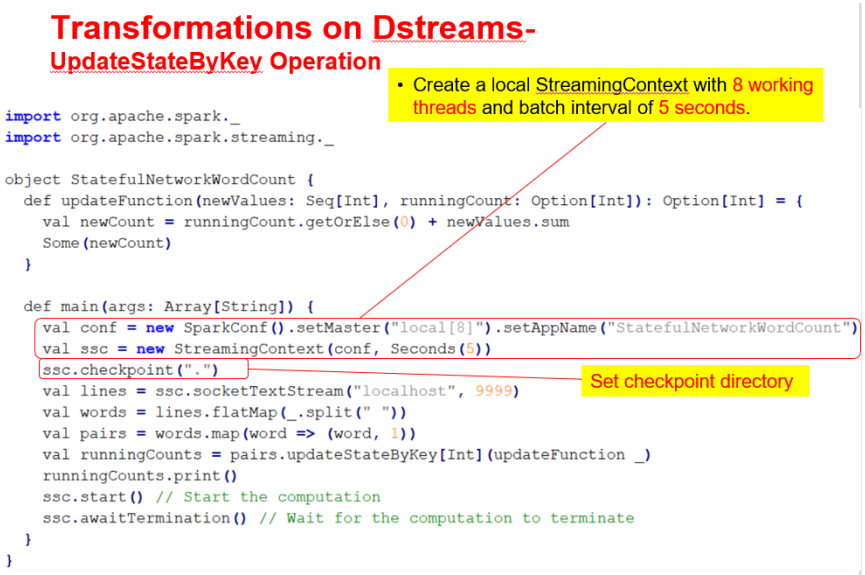
When to enable Checkpointing

Checkpointing must be enabled for applications with any of the following requirements:

When to enable Checkpointing

Checkpointing must be enabled for applications with any of the following requirements:

- Usage of stateful transformations
  - If either updateStateByKey or reduceByKeyAndWindow is used in the application, then the checkpoint directory must be provided to allow for periodic RDD checkpointing.




```
import org.apache.spark._
import org.apache.spark.streaming._

object StatefulNetworkWordCount {
  def updateFunction(newValues: Seq[Int], runningCount: Option[Int]): Option[Int] = {
    val newCount = runningCount.getOrElse(0) + newValues.sum
    Some(newCount)
  }

  def main(args: Array[String]) {
    val conf = new SparkConf().setMaster("local[8]").setAppName("StatefulNetworkWordCount")
    val ssc = new StreamingContext(conf, Seconds(5))
    ssc.checkpoint(".")
    val lines = ssc.socketTextStream("localhost", 9999)
    val words = lines.flatMap(_.split(" "))
    val pairs = words.map(word => (word, 1))
    val runningCounts = pairs.updateStateByKey[Int](updateFunction _)
    runningCounts.print()
    ssc.start() // Start the computation
    ssc.awaitTermination() // Wait for the computation to terminate
  }
}
```

When to enable Checkpointing

Checkpointing must be enabled for applications with any of the following requirements:

- Usage of stateful transformations
  - If either updateStateByKey or reduceByKeyAndWindow is used in the application, then the checkpoint directory must be provided to allow for periodic RDD checkpointing.
- Recovering from failures of the driver running the application
  - Metadata checkpoints are used to recover with progress information.

When to enable Checkpointing

Checkpointing must be enabled for applications with any of the following requirements:

- Usage of stateful transformations
  - If either updateStateByKey or reduceByKeyAndWindow is used in the application, then the checkpoint directory must be provided to allow for periodic RDD checkpointing.
- Recovering from failures of the driver running the application
  - Metadata checkpoints are used to recover with progress information.

Note that simple streaming applications without the aforementioned stateful transformations can be run without enabling checkpointing. This is often acceptable and many run Spark Streaming applications in this way.

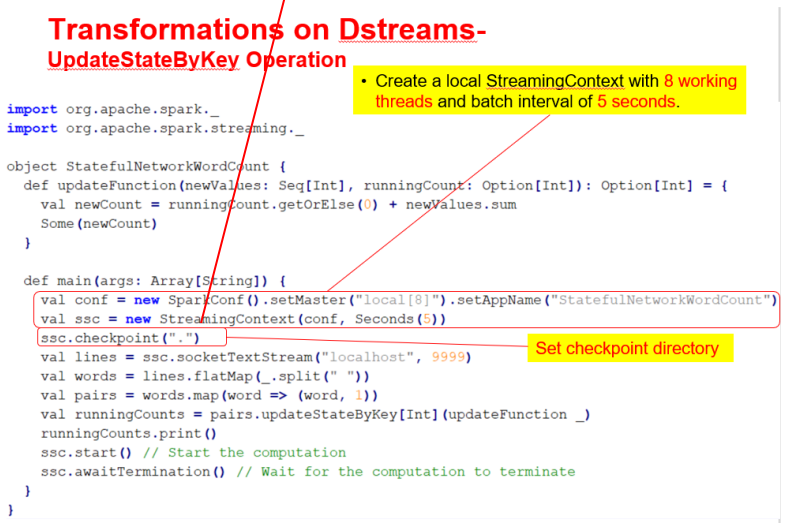
How to configure Checkpointing

Checkpointing can be enabled by setting a directory in a fault-tolerant, reliable file system (e.g., HDFS, S3, etc.) to which the checkpoint information will be saved.

How to configure Checkpointing

Checkpointing can be enabled by setting a directory in a fault-tolerant, reliable file system (e.g., HDFS, S3, etc.) to which the checkpoint information will be saved.

- This is done by using streamingContext.checkpoint(checkpointDirectory).
  - This will allow you to use the aforementioned stateful transformations.
  - If you want to make the application recover from driver failures, you should rewrite your streaming application to have the following behavior.




```
import org.apache.spark._
import org.apache.spark.streaming._

object StatefulNetworkWordCount {
  def updateFunction(newValues: Seq[Int], runningCount: Option[Int]): Option[Int] = {
    val newCount = runningCount.getOrElse(0) + newValues.sum
    Some(newCount)
  }

  def main(args: Array[String]) {
    val conf = new SparkConf().setMaster("local[8]").setAppName("StatefulNetworkWordCount")
    val ssc = new StreamingContext(conf, Seconds(5))
    ssc.checkpoint(".")
    val lines = ssc.socketTextStream("localhost", 9999)
    val words = lines.flatMap(_.split(" "))
    val pairs = words.map(word => (word, 1))
    val runningCounts = pairs.updateStateByKey[Int](updateFunction _)
    runningCounts.print()
    ssc.start() // Start the computation
    ssc.awaitTermination() // Wait for the computation to terminate
  }
}
```

Transformations on Dstreams-
UpdateStateByKey Operation

- Create a local StreamingContext with 8 working threads and batch interval of 5 seconds.

```
import org.apache.spark._
import org.apache.spark.streaming._

object StatefulNetworkWordCount {
  def updateFunction(newValues: Seq[Int], runningCount: Option[Int]): Option[Int] = {
    val newCount = runningCount.getOrElse(0) + newValues.sum
    Some(newCount)
  }

  def main(args: Array[String]) {
    val conf = new SparkConf().setMaster("local[8]").setAppName("StatefulNetworkWordCount")
    val ssc = new StreamingContext(conf, Seconds(5))
    ssc.checkpoint(".")
    val lines = ssc.socketTextStream("localhost", 9999)
    val words = lines.flatMap(_.split(" "))
    val pairs = words.map(word => (word, 1))
    val runningCounts = pairs.updateStateByKey[Int](updateFunction _)
    runningCounts.print()
    ssc.start() // Start the computation
    ssc.awaitTermination() // Wait for the computation to terminate
  }
}
```

Set checkpoint directory

How to configure Checkpointing

Checkpointing can be enabled by setting a directory in a fault-tolerant, reliable file system (e.g., HDFS, S3, etc.) to which the checkpoint information will be saved.

- This is done by using streamingContext.checkpoint(checkpointDirectory).
  - This will allow you to use the aforementioned stateful transformations.
  - If you want to make the application recover from driver failures, you should rewrite your streaming application to have the following behavior.

How to configure Checkpointing

Checkpointing can be enabled by setting a directory in a fault-tolerant, reliable file system (e.g., HDFS, S3, etc.) to which the checkpoint information will be saved.

- This is done by using streamingContext.checkpoint(checkpointDirectory).
  - This will allow you to use the aforementioned stateful transformations.
  - If you want to make the application recover from driver failures, you should rewrite your streaming application to have the following behavior.

How to configure Checkpointing

Checkpointing can be enabled by setting a directory in a fault-tolerant, reliable file system (e.g., HDFS, S3, etc.) to which the checkpoint information will be saved.

- This is done by using streamingContext.checkpoint(checkpointDirectory).
  - This will allow you to use the aforementioned stateful transformations.
  - If you want to make the application recover from driver failures, you should rewrite your streaming application to have the following behavior.
    - When the program is being started for the first time, it will create a new StreamingContext, set up all the streams and then call start().
    - When the program is being restarted after failure, it will re-create a StreamingContext from the checkpoint data in the checkpoint directory.

- When the program is being started for the first time, it will create a new StreamingContext, set up all the streams and then call start().
- When the program is being restarted after failure, it will re-create a StreamingContext from the checkpoint data in the checkpoint directory.

- When the program is being started for the first time, it will create a new StreamingContext, set up all the streams and then call start().
- When the program is being restarted after failure, it will re-create a StreamingContext from the checkpoint data in the checkpoint directory.

```
// Function to create and setup a new StreamingContext
def functionToCreateContext(): StreamingContext = {
    val ssc = new StreamingContext(...) // new context
    val lines = ssc.socketTextStream(...) // create DStreams
    ...
    ssc.checkpoint(checkpointDirectory) // set checkpoint directory
}

// Get StreamingContext from checkpoint data or create a new one
val context = StreamingContext.getOrCreate(checkpointDirectory, functionToCreateContext _)

// Do additional setup on context that needs to be done,
// irrespective of whether it is being started or restarted
context. ...

// Start the context
context.start()
context.awaitTermination()
```

- When the program is being started for the first time, it will create a new StreamingContext, set up all the streams and then call start().
- When the program is being restarted after failure, it will re-create a StreamingContext from the checkpoint data in the checkpoint directory.

```
// Function to create and setup a new StreamingContext
def functionToCreateContext(): StreamingContext = {
    val ssc = new StreamingContext(...) // new context
    val lines = ssc.socketTextStream(...) // create DStreams
    ...
    ssc.checkpoint(checkpointDirectory) // set checkpoint directory
}

// Get StreamingContext from checkpoint data or create a new one
val context = StreamingContext.getOrCreate(checkpointDirectory, functionToCreateContext _)

// Do additional setup on context that needs to be done,
// irrespective of whether it is being started or restarted
context. ...

// Start the context
context.start()
context.awaitTermination()
```

- When the program is being started for the first time, it will create a new StreamingContext, set up all the streams and then call start().
- When the program is being restarted after failure, it will re-create a StreamingContext from the checkpoint data in the checkpoint directory.

```
// Function to create and setup a new StreamingContext
def functionToCreateContext(): StreamingContext = {
    val ssc = new StreamingContext(...) // new context
    val lines = ssc.socketTextStream(...) // create DStreams
    ...
    ssc.checkpoint(checkpointDirectory) // set checkpoint directory
}

// Get StreamingContext from checkpoint data or create a new one
val context = StreamingContext.getOrCreate(checkpointDirectory, functionToCreateContext _)

// Do additional setup on context that needs to be done,
// irrespective of whether it is being started or restarted
context. ...

// Start the context
context.start()
context.awaitTermination()
```

- When the program is being started for the first time, it will create a new StreamingContext, set up all the streams and then call start().
- When the program is being restarted after failure, it will re-create a StreamingContext from the checkpoint data in the checkpoint directory.

Spark handles state and RDD restoration internally.

Shared Variables

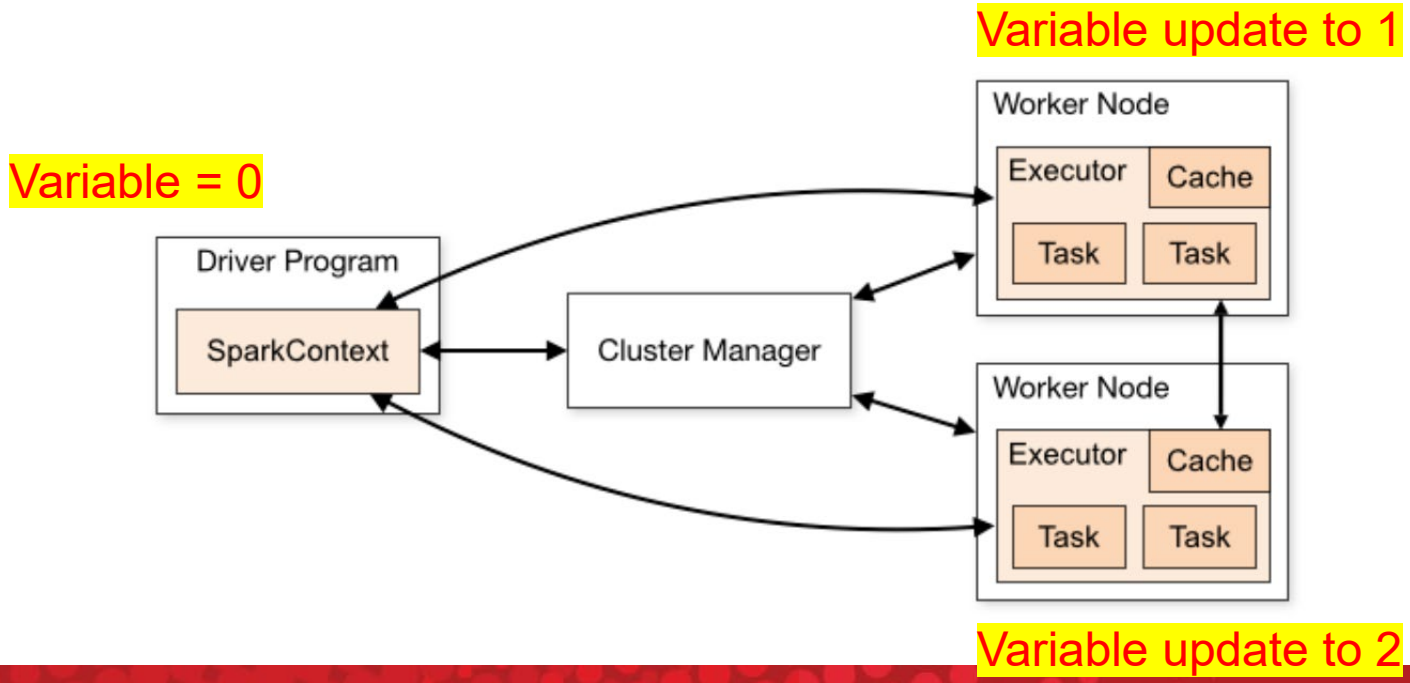
- Normally, when a function passed to a Spark operation (such as map or reduce) is executed on a remote cluster node, it works on separate copies of all the variables used in the function.
- These variables are copied to each machine, and no updates to the variables on the remote machine are propagated back to the driver program.

Shared Variables

- Normally, when a function passed to a Spark operation (such as map or reduce) is executed on a remote cluster node, it works on separate copies of all the variables used in the function.
- These variables are copied to each machine, and no updates to the variables on the remote machine are propagated back to the driver program.

Shared Variables

- Normally, when a function passed to a Spark operation (such as map or reduce) is executed on a remote cluster node, it works on separate copies of all the variables used in the function.
- These variables are copied to each machine, and no updates to the variables on the remote machine are propagated back to the driver program.



Shared Variables

- Normally, when a function passed to a Spark operation (such as map or reduce) is executed on a remote cluster node, it works on separate copies of all the variables used in the function.
- These variables are copied to each machine, and no updates to the variables on the remote machine are propagated back to the driver program.

Supporting general, read-write shared variables across tasks would be inefficient. However, Spark does provide two limited types of shared variables for two common usage patterns:

- Broadcast variables
- Accumulators.

Shared Variables

- Normally, when a function passed to a Spark operation (such as map or reduce) is executed on a remote cluster node, it works on separate copies of all the variables used in the function.
- These variables are copied to each machine, and no updates to the variables on the remote machine are propagated back to the driver program.

Supporting general, read-write shared variables across tasks would be inefficient. However, Spark does provide two limited types of shared variables for two common usage patterns:

- Broadcast variables
- Accumulators.

Launching Applications with spark-submit

```
./bin/spark-submit \
  --class <main-class> \
  --master <master-url> \
  --deploy-mode <deploy-mode> \
  --conf <key>=<value> \
  ... # other options
  <application-jar> \
  [application-arguments]
```

```
[hadoop@ip-192-168-18-110 ~]$ spark-submit --class streaming.count.NetworkW
ordCount --master yarn --deploy-mode client count-0.0.1-SNAPSHOT-jar-with-d
ependencies.jar hdfs:///user/ke
```

Launching Applications with spark-submit

```
./bin/spark-submit \
  --class <main-class> \
  --master <master-url> \
  --deploy-mode <deploy-mode> \
  --conf <key>=<value> \
  ... # other options
  <application-jar> \
  [application-arguments]
```

The entry point for your application (e.g. org.apache.spark.examples.SparkPi)

```
[hadoop@ip-192-168-18-110 ~]$ spark-submit --class streaming.count.NetworkW
ordCount --master yarn --deploy-mode client count-0.0.1-SNAPSHOT-jar-with-d
ependencies.jar hdfs:///user/ke
```

Launching Applications with spark-submit

```
./bin/spark-submit \
  --class <main-class> \
  --master <master-url> \
  --deploy-mode <deploy-mode> \
  --conf <key>=<value> \
  ... # other options
  <application-jar> \
  [application-arguments]
```

The entry point for your application (e.g. org.apache.spark.examples.SparkPi)

The master URL for the cluster (e.g. spark://23.195.26.187:7077)

Run on a YARN cluster

```
[hadoop@ip-192-168-18-110 ~]$ spark-submit --class streaming.count.NetworkW
ordCount --master yarn --deploy-mode client count-0.0.1-SNAPSHOT-jar-with-d
ependencies.jar hdfs:///user/ke
```

Launching Applications with spark-submit

```
./bin/spark-submit \
  --class <main-class> \
  --master <master-url> \
  --deploy-mode <deploy-mode> \
  --conf <key>=<value> \
  ... # other options
  <application-jar> \
  [application-arguments]
```

The entry point for your application (e.g. org.apache.spark.examples.SparkPi)

The master URL for the cluster (e.g. spark://23.195.26.187:7077)

Whether to deploy your driver on the worker nodes (cluster) or locally as an external client (client) (default: client)

```
[hadoop@ip-192-168-18-110 ~]$ spark-submit --class streaming.count.NetworkW
ordCount --master yarn --deploy-mode client count-0.0.1-SNAPSHOT-jar-with-d
ependencies.jar hdfs:///user/ke
```

Launching Applications with spark-submit

```
./bin/spark-submit \
  --class <main-class> \
  --master <master-url> \
  --deploy-mode <deploy-mode> \
  --conf <key>=<value> \
  ... # other options
  <application-jar> \
  [application-arguments]
```

The entry point for your application (e.g. org.apache.spark.examples.SparkPi)

The master URL for the cluster (e.g. spark://23.195.26.187:7077)

Whether to deploy your driver on the worker nodes (cluster) or locally as an external client (client) (default: client)

Arbitrary Spark configuration property in key=value format.

```
[hadoop@ip-192-168-18-110 ~]$ spark-submit --class streaming.count.NetworkW
ordCount --master yarn --deploy-mode client count-0.0.1-SNAPSHOT-jar-with-d
ependencies.jar hdfs:///user/ke
```

Launching Applications with spark-submit

```
./bin/spark-submit \
  --class <main-class> \
  --master <master-url> \
  --deploy-mode <deploy-mode> \
  --conf <key>=<value> \
  ... # other options
  <application-jar> \
  [application-arguments]
```

The entry point for your application (e.g. org.apache.spark.examples.SparkPi)

The master URL for the cluster (e.g. spark://23.195.26.187:7077)

Whether to deploy your driver on the worker nodes (cluster) or locally as an external client (client) (default: client)

Arbitrary Spark configuration property in key=value format.

Path to a standalone jar including your application and all dependencies.

```
[hadoop@ip-192-168-18-110 ~]$ spark-submit --class streaming.count.NetworkW
ordCount --master yarn --deploy-mode client count-0.0.1-SNAPSHOT-jar-with-d
ependencies.jar hdfs:///user/ke
```

Launching Applications with spark-submit

```
./bin/spark-submit \
  --class <main-class> \
  --master <master-url> \
  --deploy-mode <deploy-mode> \
  --conf <key>=<value> \
  ... # other options
  <application-jar> \
  [application-arguments]
```

The entry point for your application (e.g. org.apache.spark.examples.SparkPi)

The master URL for the cluster (e.g. spark://23.195.26.187:7077)

Whether to deploy your driver on the worker nodes (cluster) or locally as an external client (client) (default: client)

Arbitrary Spark configuration property in key=value format.

Path to a standalone jar including your application and all dependencies.

Arguments passed to the main method of your main class, if any

```
[hadoop@ip-192-168-18-110 ~]$ spark-submit --class streaming.count.NetworkW
ordCount --master yarn --deploy-mode client count-0.0.1-SNAPSHOT-jar-with-d
ependencies.jar hdfs:///user/ke
```

Q6 - Simple streaming applications without stateful transformations can be run without enabling checkpointing.

- True
- False

Q6 - Simple streaming applications without stateful transformations can be run without enabling checkpointing.

- ✓ True
- ○ False

Q7 - Regarding pub-sub messaging system, which statement is correct?

- The producer knows the destination of the message and posts the message directly to the consumer's queue.
- Only one consumer will get the message.
- Subscribers can decide to remain continuously active to receive messages or pause for a moment.
- Neither the publisher nor the subscriber know about each other. A good metaphor for this model is the anonymous bulletin board.

Spark Streaming Input and Output

Spark Streaming

- inputs from various reliable inputs sources like Flume, HDFS, and Kafka etc.
- outputs the processed data to filesystems, database or live dashboards.



Apache Kafka

In Big Data, an enormous volume of data is used. Regarding data, we have two main challenges.

- The first challenge is how to collect large volume of data
- The second challenge is to analyze the collected data.

To overcome those challenges, you must need a messaging system.

Apache Kafka

In Big Data, an enormous volume of data is used. Regarding data, we have two main challenges.

- The first challenge is how to collect large volume of data
- The second challenge is to analyze the collected data.

To overcome those challenges, you must need a messaging system.

Apache Kafka

In Big Data, an enormous volume of data is used.
Regarding data, we have two main challenges.

- The first challenge is how to collect large volume of data
- The second challenge is to analyze the collected data.

To overcome those challenges, you must need a messaging system.

What is a Messaging System?

- A Messaging System is responsible for transferring data from one application to another, so the applications can focus on data, but not worry about how to share it.
  - Distributed messaging is based on the concept of reliable message queuing.
  - Messages are queued asynchronously between client applications and messaging system.
- Two types of messaging patterns are available:
  - point to point
  - publish-subscribe (pub-sub) messaging system. Most of the messaging patterns follow **pub-sub**.

What is a Messaging System?

- A Messaging System is responsible for transferring data from one application to another, so the applications can focus on data, but not worry about how to share it.
  - Distributed messaging is based on the concept of reliable message queuing.
  - Messages are queued asynchronously between client applications and messaging system.
- Two types of messaging patterns are available:
  - point to point
  - publish-subscribe (pub-sub) messaging system. Most of the messaging patterns follow **pub-sub**.

What is a Messaging System?

- A Messaging System is responsible for transferring data from one application to another, so the applications can focus on data, but not worry about how to share it.
  - Distributed messaging is based on the concept of reliable message queuing.
  - Messages are queued asynchronously between client applications and messaging system.
- Two types of messaging patterns are available:
  - point to point
  - publish-subscribe (pub-sub) messaging system. Most of the messaging patterns follow **pub-sub**.

What is a Messaging System?

- A Messaging System is responsible for transferring data from one application to another, so the applications can focus on data, but not worry about how to share it.
  - Distributed messaging is based on the concept of reliable message queuing.
  - Messages are queued asynchronously between client applications and messaging system.
- Two types of messaging patterns are available:
  - point to point
  - publish-subscribe (pub-sub) messaging system. Most of the messaging patterns follow **pub-sub**.

What is a Messaging System?

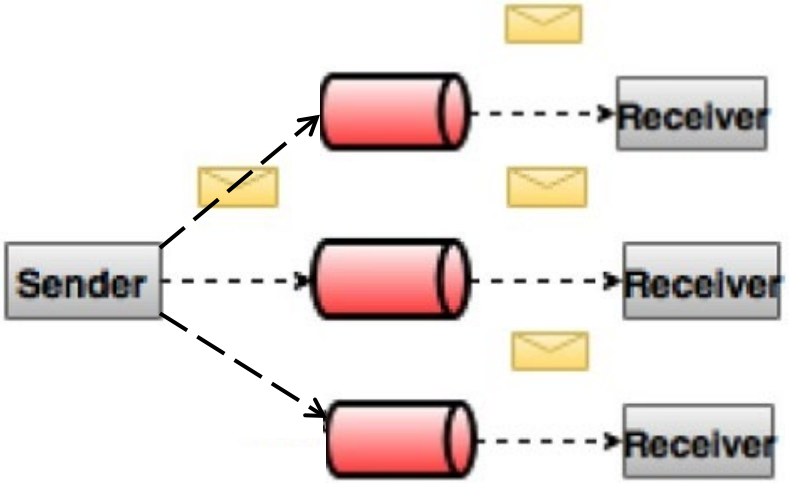
- A Messaging System is responsible for transferring data from one application to another, so the applications can focus on data, but not worry about how to share it.
  - Distributed messaging is based on the concept of reliable message queuing.
  - Messages are queued asynchronously between client applications and messaging system.
- Two types of messaging patterns are available:
  - point to point
  - publish-subscribe (pub-sub) messaging system. Most of the messaging patterns follow **pub-sub**.

What is a Messaging System?

- A Messaging System is responsible for transferring data from one application to another, so the applications can focus on data, but not worry about how to share it.
  - Distributed messaging is based on the concept of reliable message queuing.
  - Messages are queued asynchronously between client applications and messaging system.
- Two types of messaging patterns are available:
  - point to point
  - publish-subscribe (pub-sub) messaging system. Most of the messaging patterns follow **pub-sub**.

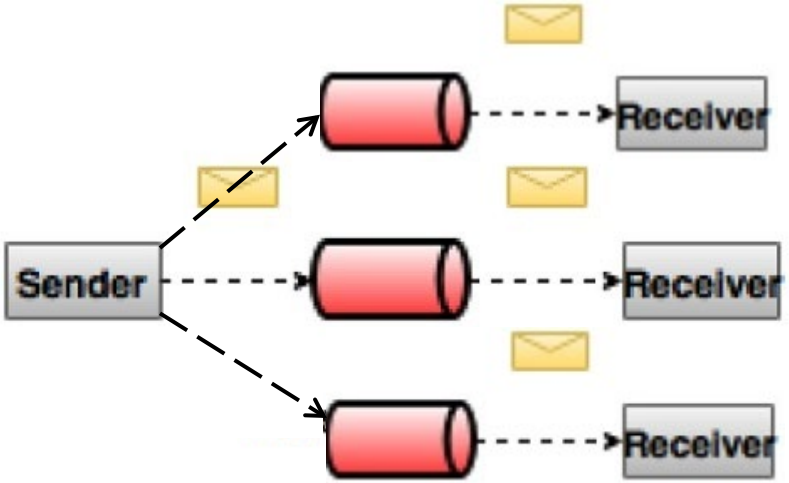
Point to Point Messaging System

- In the point-to-point model, a producer posts messages to a particular queue and a consumer reads messages from the queue.
  - Here, the producer knows the destination of the message and posts the message directly to the consumer's queue.



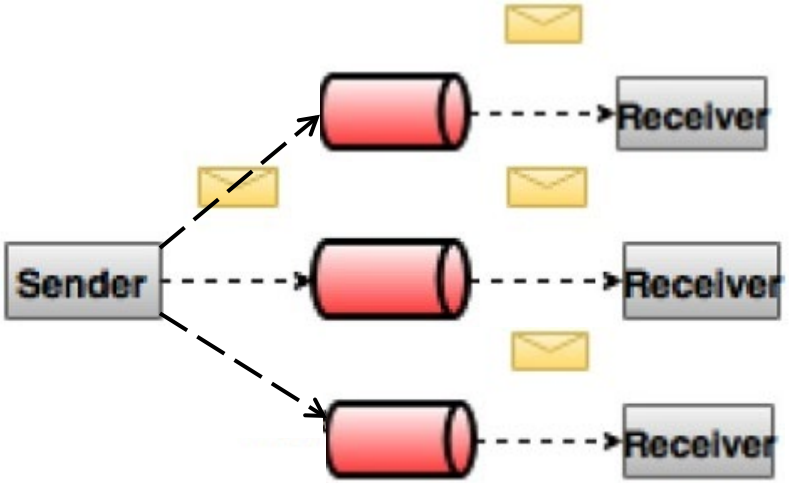
Point to Point Messaging System

- In the point-to-point model, a producer posts messages to a particular queue and a consumer reads messages from the queue.
  - Here, the producer knows the destination of the message and posts the message directly to the consumer's queue.



Point to Point Messaging System

- In the point-to-point model, a producer posts messages to a particular queue and a consumer reads messages from the queue.
  - Here, the producer knows the destination of the message and posts the message directly to the consumer's queue.

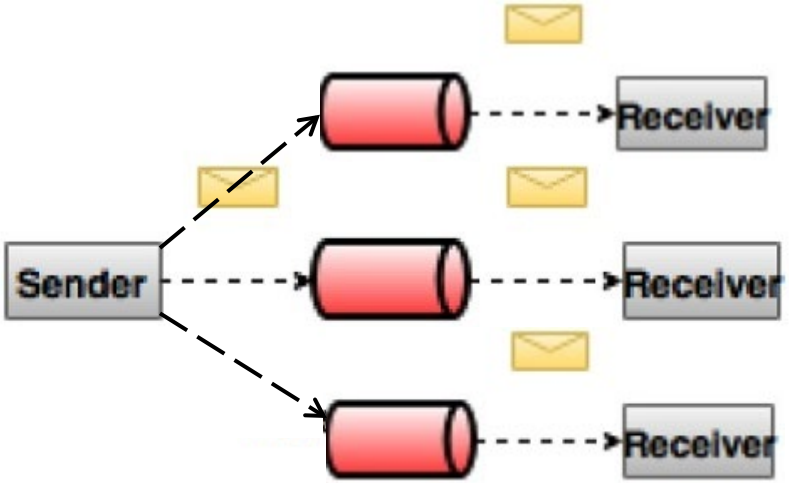


The point-to-point model has the following characteristics:

- Only one consumer will get the message.
- The producer does not have to be running at the time the receiver consumes the message, nor does the receiver need to be running at the time the message is sent.
- Every message successfully processed is acknowledged by the receiver.

Point to Point Messaging System

- In the point-to-point model, a producer posts messages to a particular queue and a consumer reads messages from the queue.
  - Here, the producer knows the destination of the message and posts the message directly to the consumer's queue.

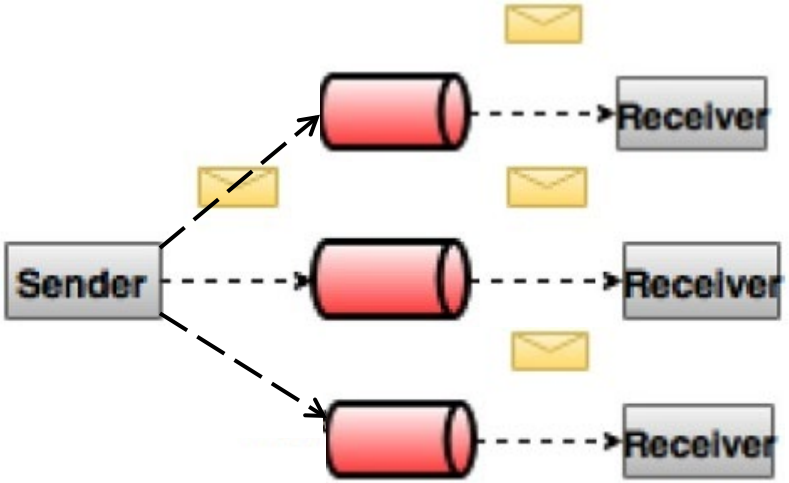


The point-to-point model has the following characteristics:

- Only one consumer will get the message.
- The producer does not have to be running at the time the receiver consumes the message, nor does the receiver need to be running at the time the message is sent.
- Every message successfully processed is acknowledged by the receiver.

Point to Point Messaging System

- In the point-to-point model, a producer posts messages to a particular queue and a consumer reads messages from the queue.
  - Here, the producer knows the destination of the message and posts the message directly to the consumer's queue.



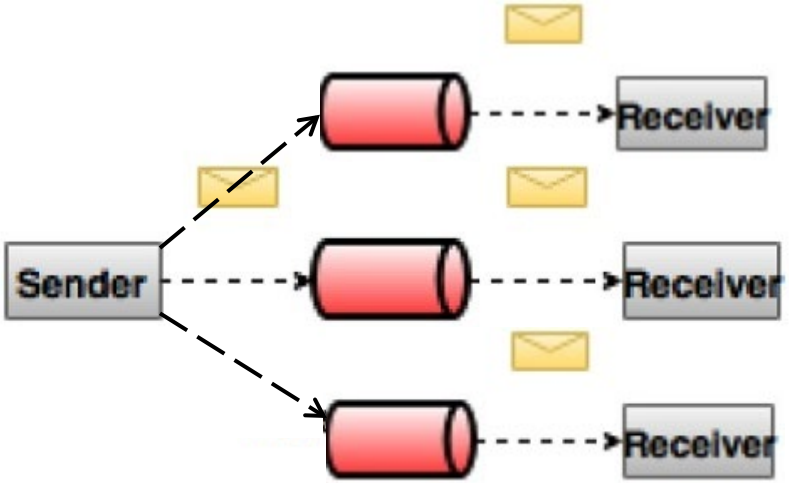
asynchronous

The point-to-point model has the following characteristics:

- Only one consumer will get the message.
- The producer does not have to be running at the time the receiver consumes the message, nor does the receiver need to be running at the time the message is sent.
- Every message successfully processed is acknowledged by the receiver.

Point to Point Messaging System

- In the point-to-point model, a producer posts messages to a particular queue and a consumer reads messages from the queue.
  - Here, the producer knows the destination of the message and posts the message directly to the consumer's queue.

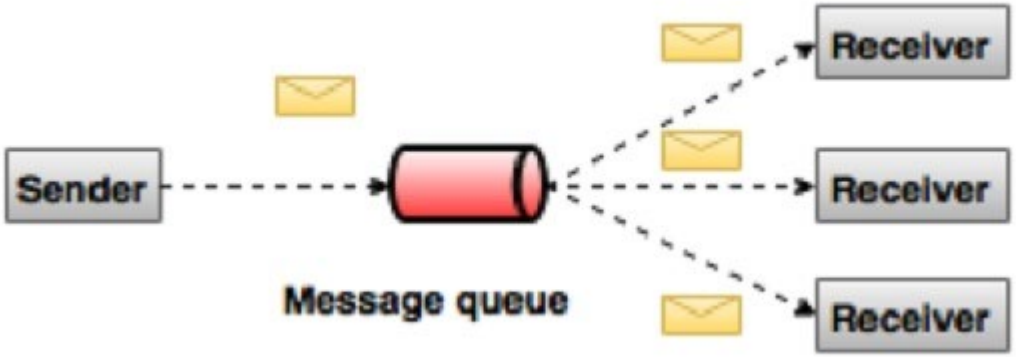


The point-to-point model has the following characteristics:

- Only one consumer will get the message.
- The producer does not have to be running at the time the receiver consumes the message, nor does the receiver need to be running at the time the message is sent.
- Every message successfully processed is acknowledged by the receiver.

Publish-Subscribe Messaging System

- The publish-subscribe model supports publishing messages to a particular message topic. Zero or more subscribers may register interest in receiving messages on a particular message topic.
  - Neither the publisher nor the subscriber know about each other.
  - A good metaphor for this model is the anonymous bulletin board.

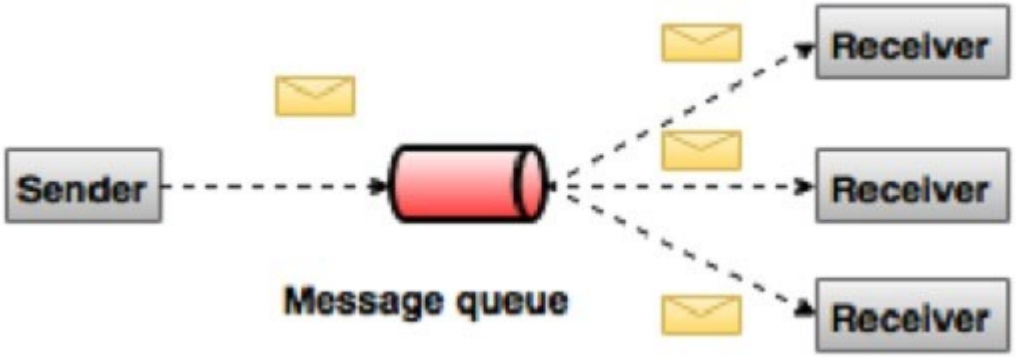


The publish and subscribe model has the following characteristics:

- Multiple consumers can receive messages.
- A timing dependency exists between publishers and subscribers.
  - The publisher has to create a subscription before clients can subscribe. Once subscribed, subscribers have to remain continuously active to receive messages

Publish-Subscribe Messaging System

- The publish-subscribe model supports publishing messages to a particular message topic. Zero or more subscribers may register interest in receiving messages on a particular message topic.
  - Neither the publisher nor the subscriber know about each other.
  - A good metaphor for this model is the anonymous bulletin board.

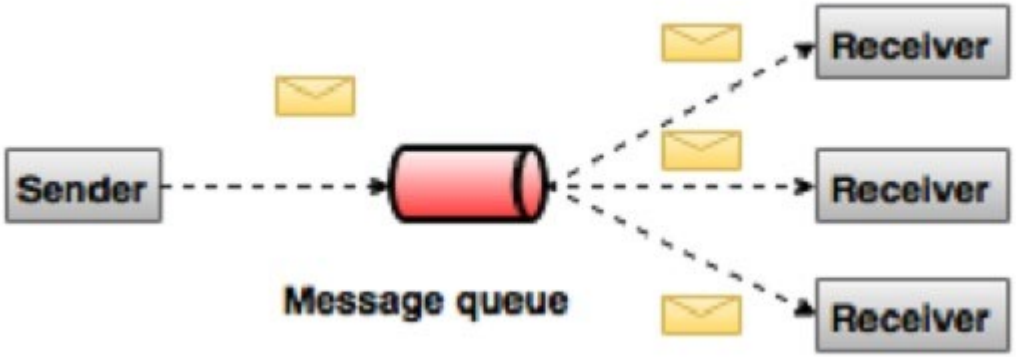


The publish and subscribe model has the following characteristics:

- Multiple consumers can receive messages.
- A timing dependency exists between publishers and subscribers.
  - The publisher has to create a subscription before clients can subscribe. Once subscribed, subscribers have to remain continuously active to receive messages

Publish-Subscribe Messaging System

- The publish-subscribe model supports publishing messages to a particular message topic. Zero or more subscribers may register interest in receiving messages on a particular message topic.
  - Neither the publisher nor the subscriber know about each other.
  - A good metaphor for this model is the anonymous bulletin board.

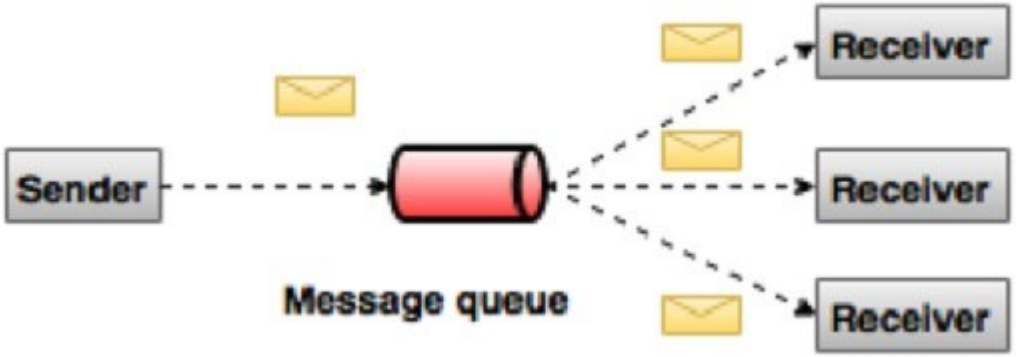


The publish and subscribe model has the following characteristics:

- Multiple consumers can receive messages.
- A timing dependency exists between publishers and subscribers.
  - The publisher has to create a subscription before clients can subscribe. Once subscribed, subscribers have to remain continuously active to receive messages

Publish-Subscribe Messaging System

- The publish-subscribe model supports publishing messages to a particular message topic. Zero or more subscribers may register interest in receiving messages on a particular message topic.
  - Neither the publisher nor the subscriber know about each other.
  - A good metaphor for this model is the anonymous bulletin board.



The publish and subscribe model has the following characteristics:

- Multiple consumers can receive messages.
- A timing dependency exists between publishers and subscribers.
  - The publisher has to create a subscription before clients can subscribe. Once subscribed, subscribers have to remain continuously active to receive messages

Publish-Subscribe Messaging System

The main reason is that basic pub-sub (or non-durable pub-sub) is designed for **real-time broadcasting**, not for storing messages separately for every subscriber.

When a publisher sends a message:
Publisher → Broker → currently active subscribers

If Subscriber B is offline when the message is published, the broker may simply discard that message after delivering it to the active subscribers.

The publish and subscribe model has the following characteristics:

- Multiple consumers can receive messages.
- A timing dependency exists between publishers and subscribers.
  - The publisher has to create a subscription before clients can subscribe. Once subscribed, subscribers have to remain continuously active to receive messages

What is Kafka?

- Apache Kafka is a distributed publish-subscribe messaging system
- Kafka is a unified platform for handling all the real-time data feeds.
  - Kafka supports low latency message delivery and gives guarantee for fault tolerance in the presence of machine failures.
  - It has the ability to handle a large number of diverse consumers.
    - Kafka is very fast, performs 2 million writes/sec.

Kafka supports publish–subscribe with durable storage and replay – so does not require consumer always active, but if it stays offline too long and those messages are deleted by the retention policy, it cannot recover them from Kafka.

What is Kafka?

- Apache Kafka is a distributed publish-subscribe messaging system
- Kafka is a unified platform for handling all the real-time data feeds.
  - Kafka supports low latency message delivery and gives guarantee for fault tolerance in the presence of machine failures.
  - It has the ability to handle a large number of diverse consumers.
    - Kafka is very fast, performs 2 million writes/sec.

Kafka supports publish–subscribe with durable storage and replay – so does not require consumer always active, but if it stays offline too long and those messages are deleted by the retention policy, it cannot recover them from Kafka.

What is Kafka?

- Apache Kafka is a distributed publish-subscribe messaging system
- Kafka is a unified platform for handling all the real-time data feeds.
  - Kafka supports low latency message delivery and gives guarantee for fault tolerance in the presence of machine failures.
  - It has the ability to handle a large number of diverse consumers.
    - Kafka is very fast, performs 2 million writes/sec.

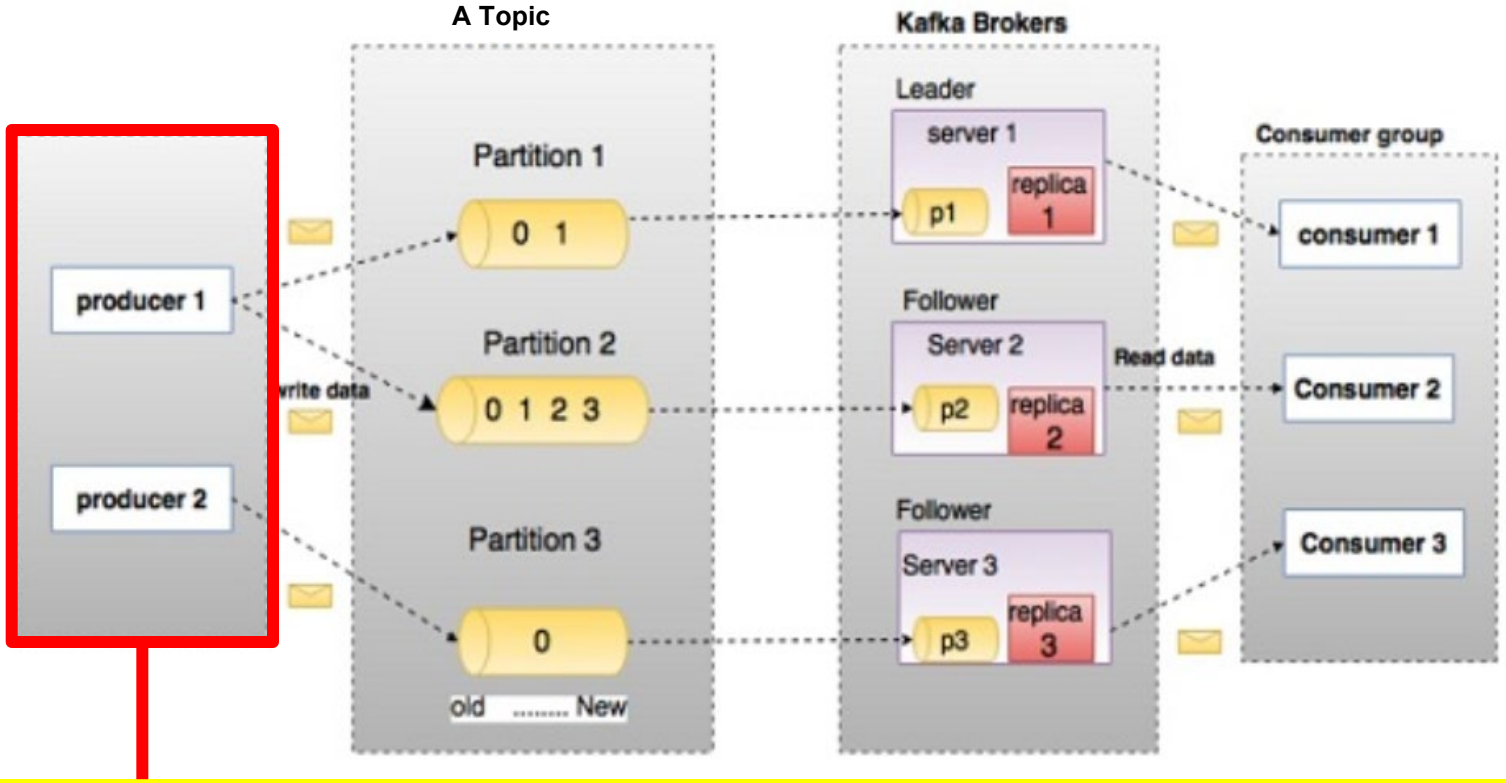
Kafka supports publish–subscribe with durable storage and replay – so does not require consumer always active, but if it stays offline too long and those messages are deleted by the retention policy, it cannot recover them from Kafka.

What is Kafka?

- Apache Kafka is a distributed publish-subscribe messaging system
- Kafka is a unified platform for handling all the real-time data feeds.
  - Kafka supports low latency message delivery and gives guarantee for fault tolerance in the presence of machine failures.
  - It has the ability to handle a large number of diverse consumers.
    - Kafka is very fast, performs 2 million writes/sec.

Kafka supports publish–subscribe with durable storage and replay – so does not require consumer always active, but if it stays offline too long and those messages are deleted by the retention policy, it cannot recover them from Kafka.

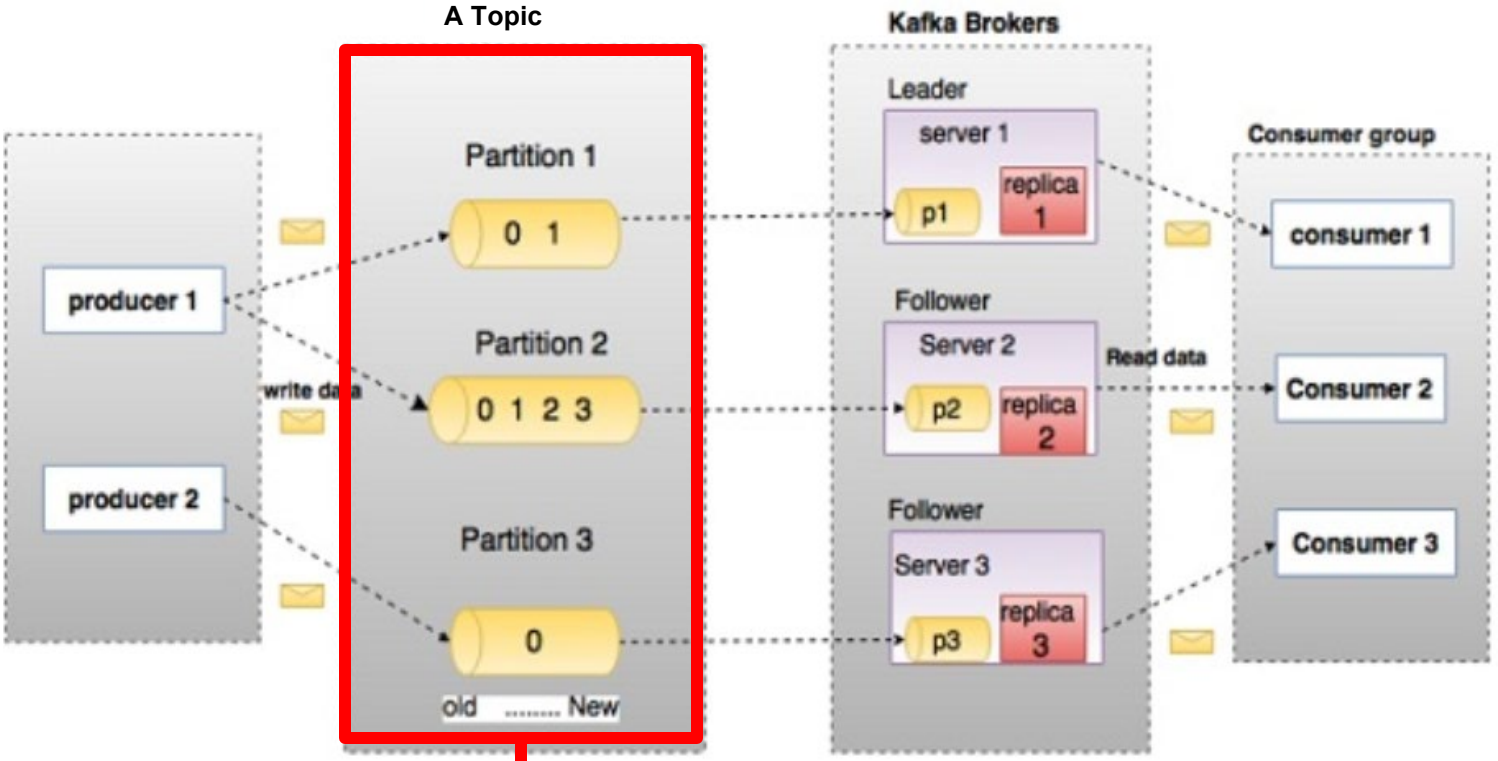
Apache Kafka - Fundamentals



Producers

- Producers are the publisher of messages to one or more Kafka topics.

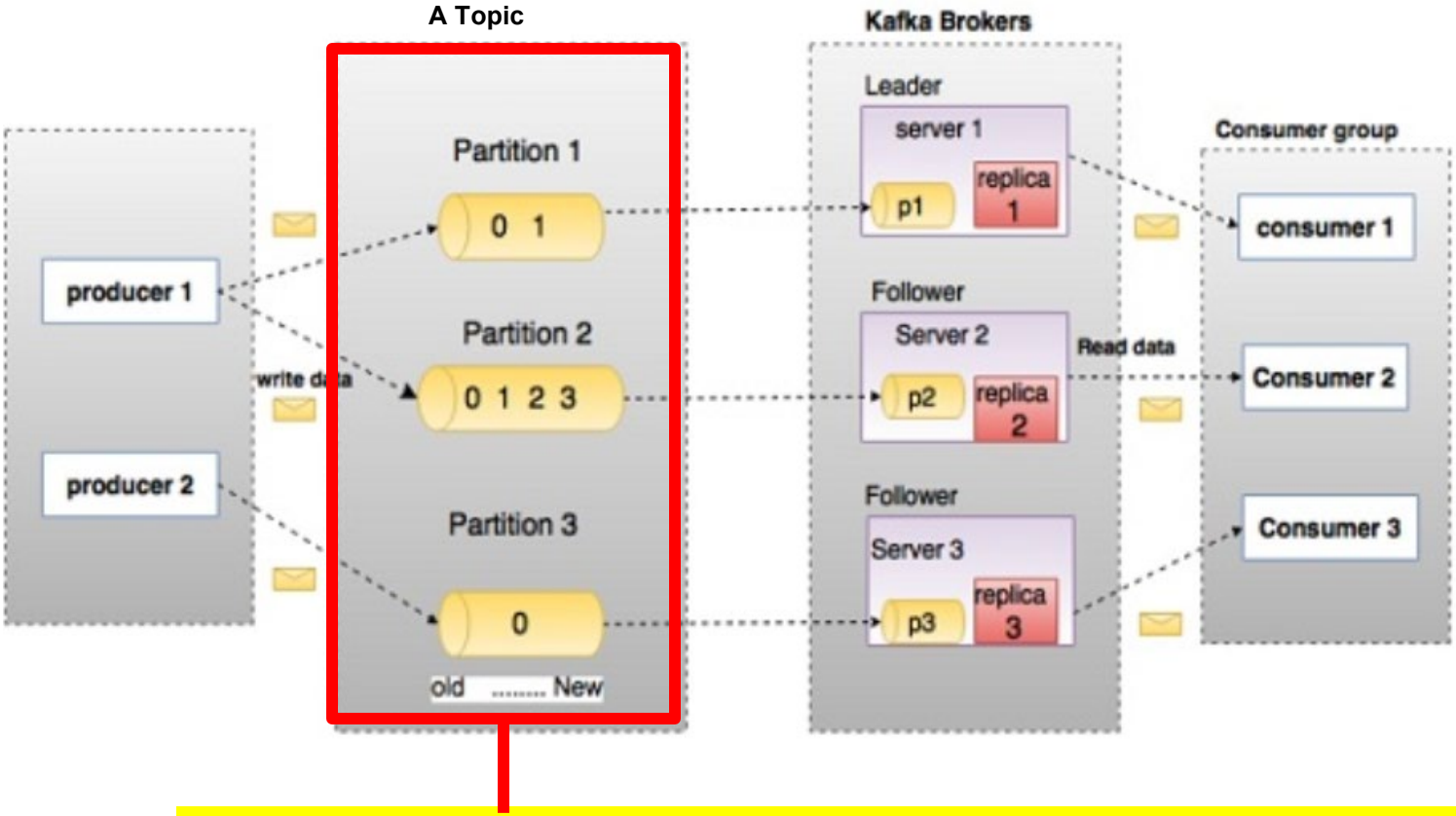
Apache Kafka - Fundamentals



Topics

- A stream of messages belonging to a particular category is called a topic. Data is stored in topics.
- A topic is split into partitions.
  - Kafka spreads partitions across multiple servers or disks;
  - So it can handle an arbitrary amount of data.

Apache Kafka - Fundamentals



Partition offset

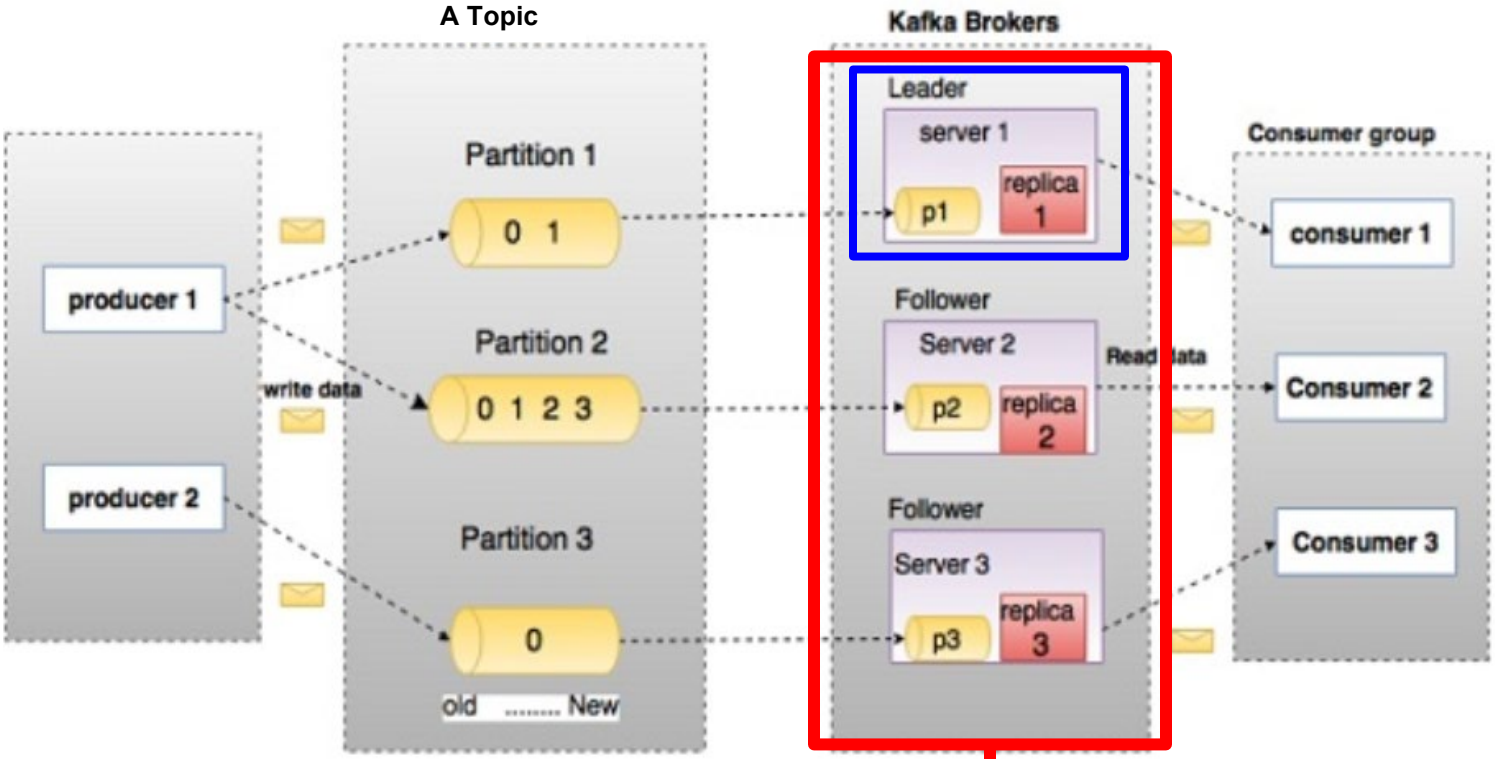
Each partitioned message has a unique sequence id called as offset. For example:

- Partition 1 has two offset factors 0 and 1.
- Partition 2 has four offset factors 0, 1, 2, and 3.
- Partition 3 has one offset factor 0.

RMIT Classification: Trusted

Apache Kafka

A **Kafka server**, a **Kafka broker** and a **Kafka node** all refer to the same concept and are synonyms (see the scaladoc of KafkaServer).



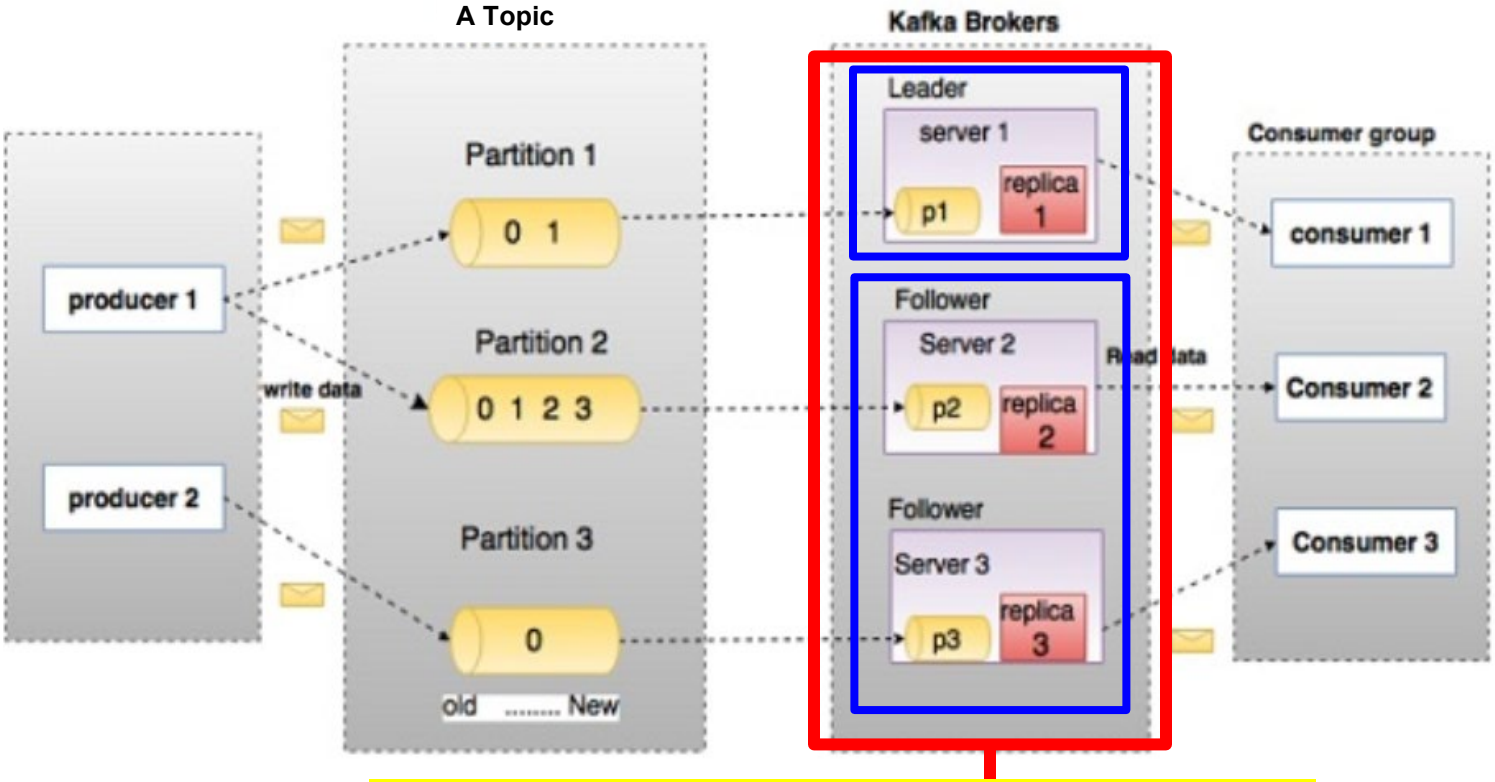
one server which acts as the leader

- the leader handles all read and write requests for the partition

Each partition has

Kafka supports publish–subscribe with durable storage and replay – so does not require consumer always active, but if it stays offline too long and those messages are deleted by the retention policy, it cannot recover them from Kafka.

Apache Kafka - Fundamentals

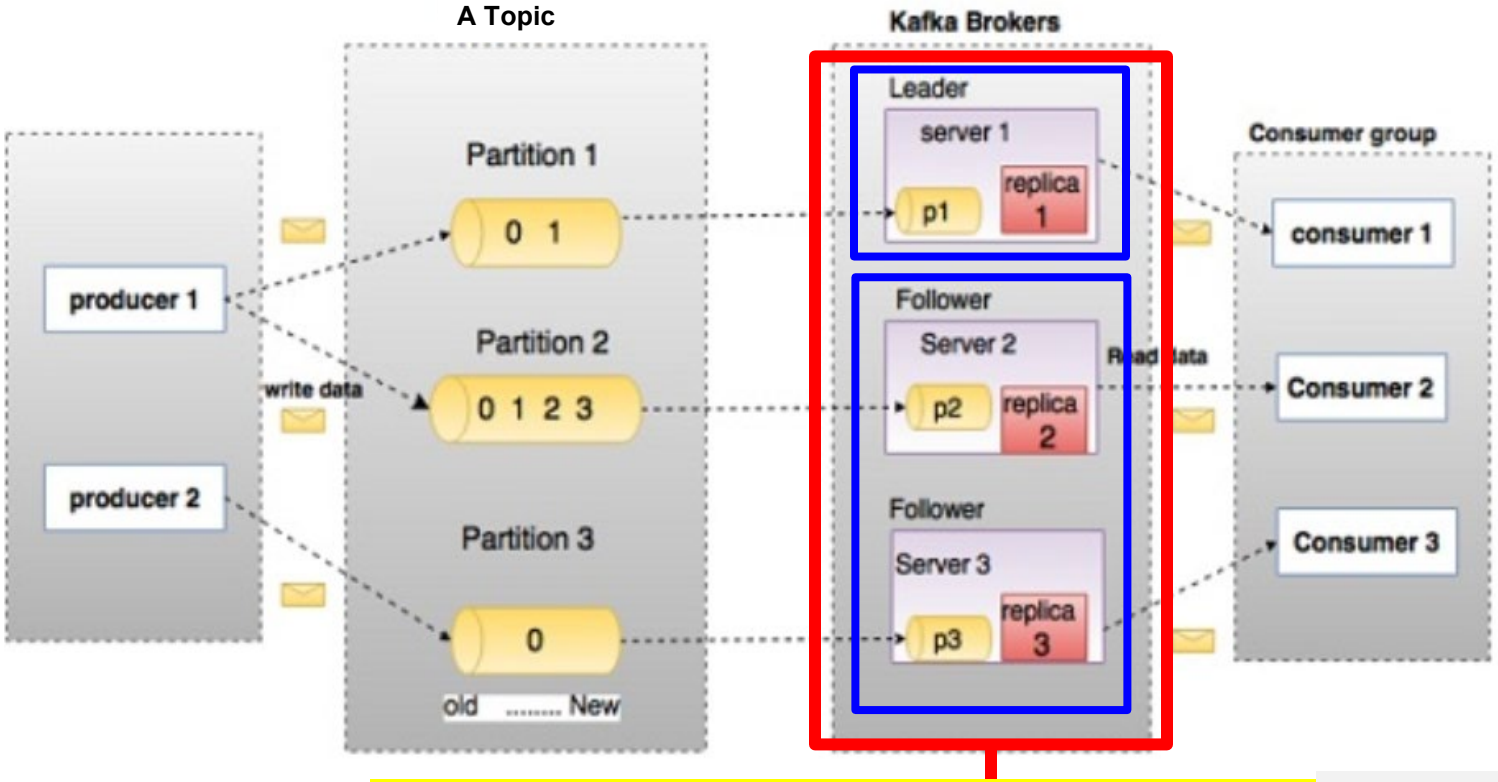


Each partition has

one server which acts as the leader
- the leader handles all read and write requests for the partition

zero or more servers which act as followers.
- followers replicate the leader
- If the leader fails, one of the followers will automatically become the new leader

Apache Kafka - Fundamentals



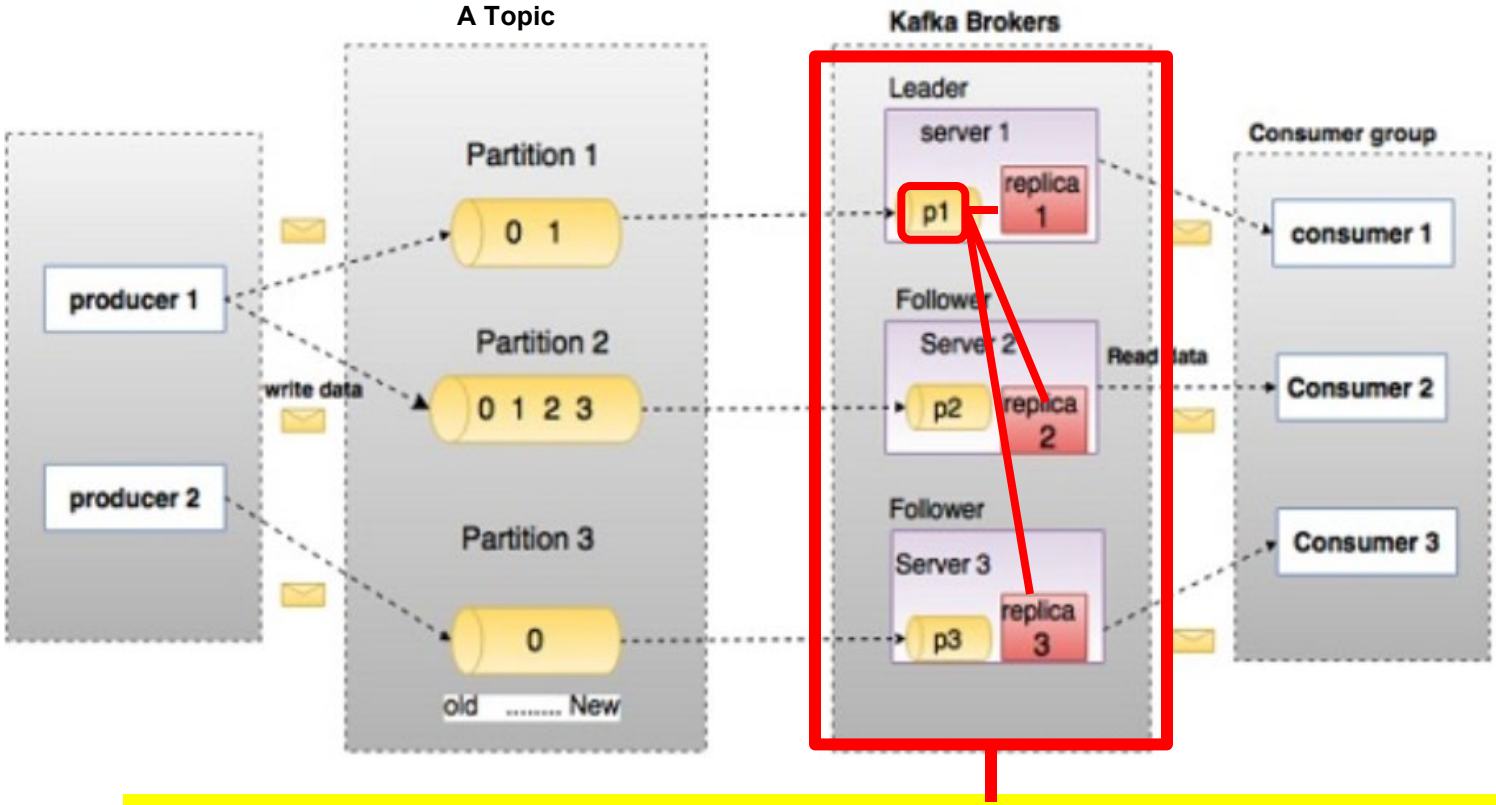
Each partition has

one server which acts as the leader
- the leader handles all read and write requests for the partition

zero or more servers which act as followers.
- followers replicate the leader
- If the leader fails, one of the followers will automatically become the new leader

Each server
- acts as a leader for some of its partitions and
- acts as a follower for others so load is well balanced within the cluster.

Apache Kafka - Fundamentals



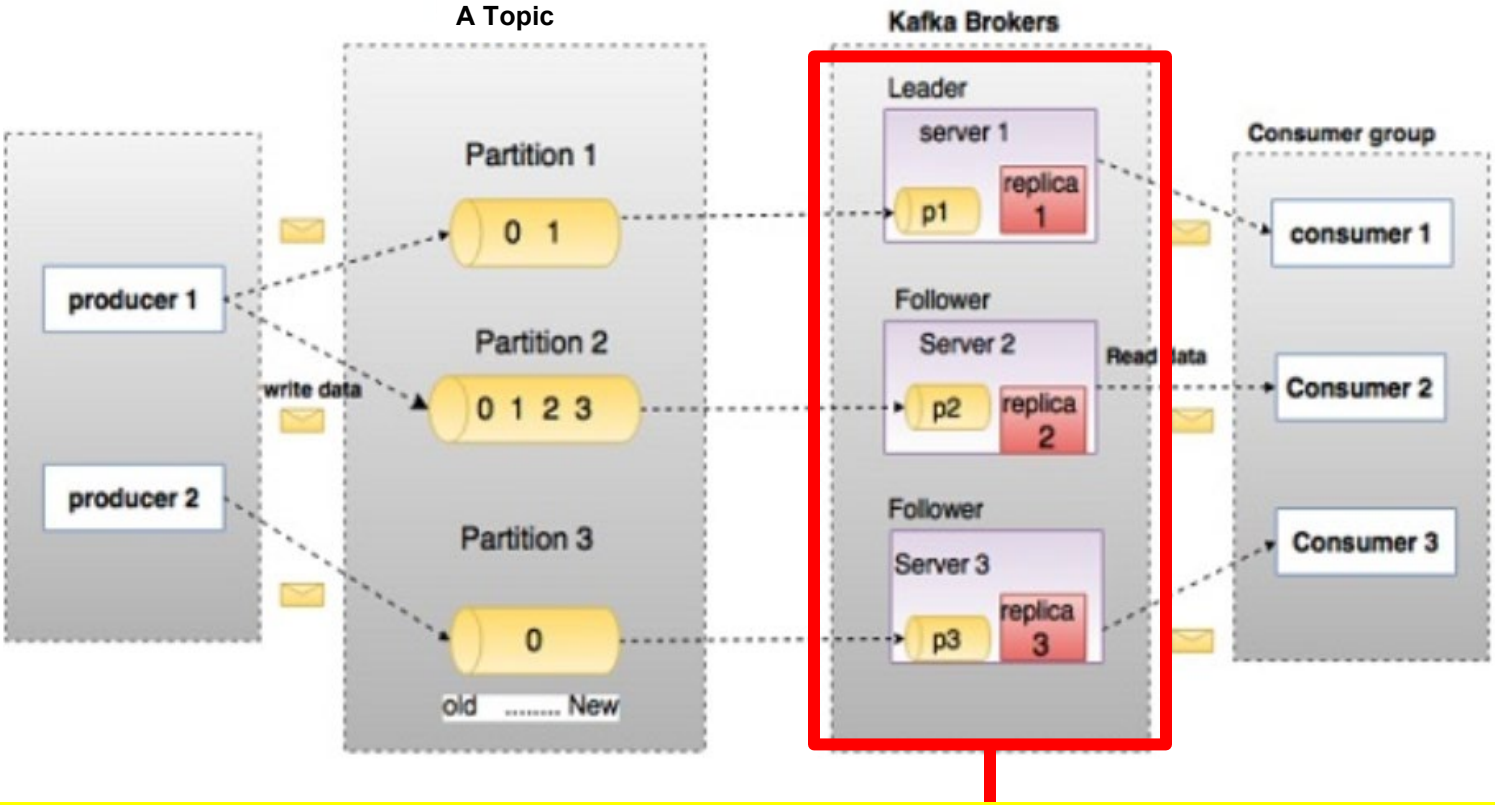
Replicas of partition

Replicas are nothing but backups of a partition. Replicas are never read or write data. They are used to prevent data loss.

Assume the replication factor of the topic is set to 3,

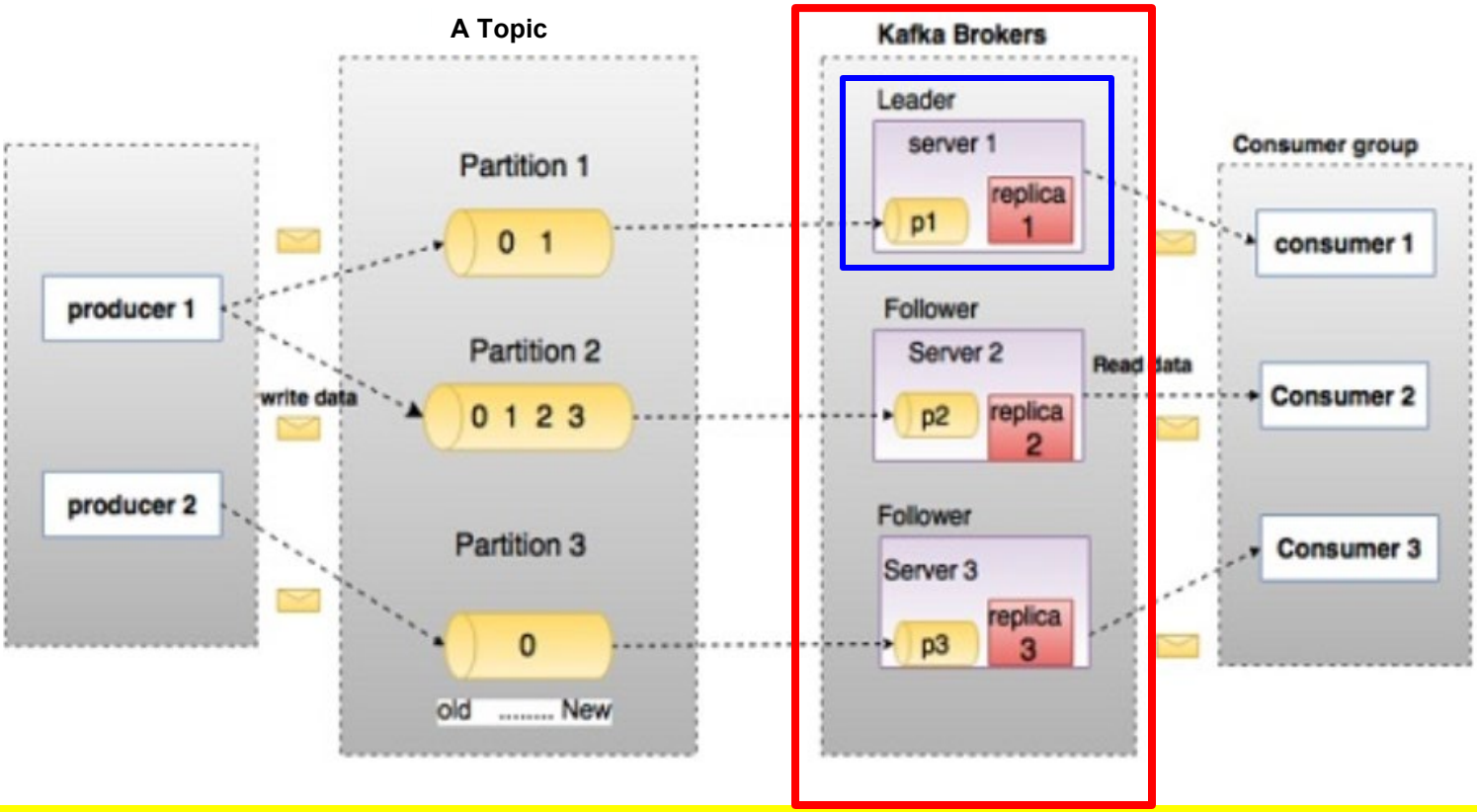
- Kafka will create 3 identical replicas of each partition and place them in the cluster to make available for all its operations.
- To balance a load in cluster, each broker stores one or more of those partitions.

Apache Kafka - Fundamentals



A Kafka cluster consists of one or more servers (Kafka brokers) running Kafka

Apache Kafka - Fundamentals

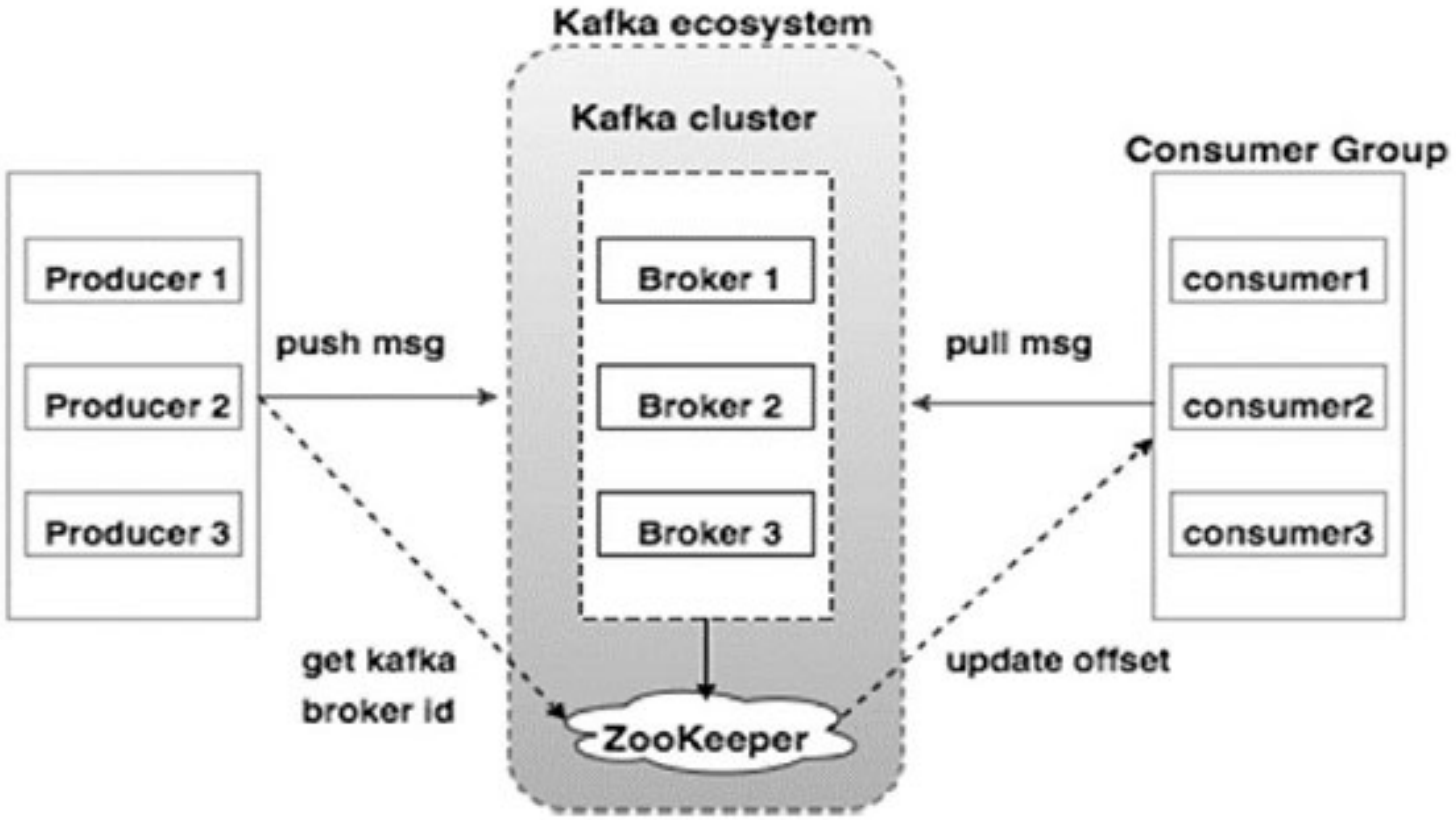


A Kafka cluster consists of one or more servers (Kafka brokers) running Kafka

Broker

- Kafka cluster typically consists of multiple brokers to maintain load balance.
- Kafka brokers are stateless, so they use ZooKeeper for maintaining their cluster state.

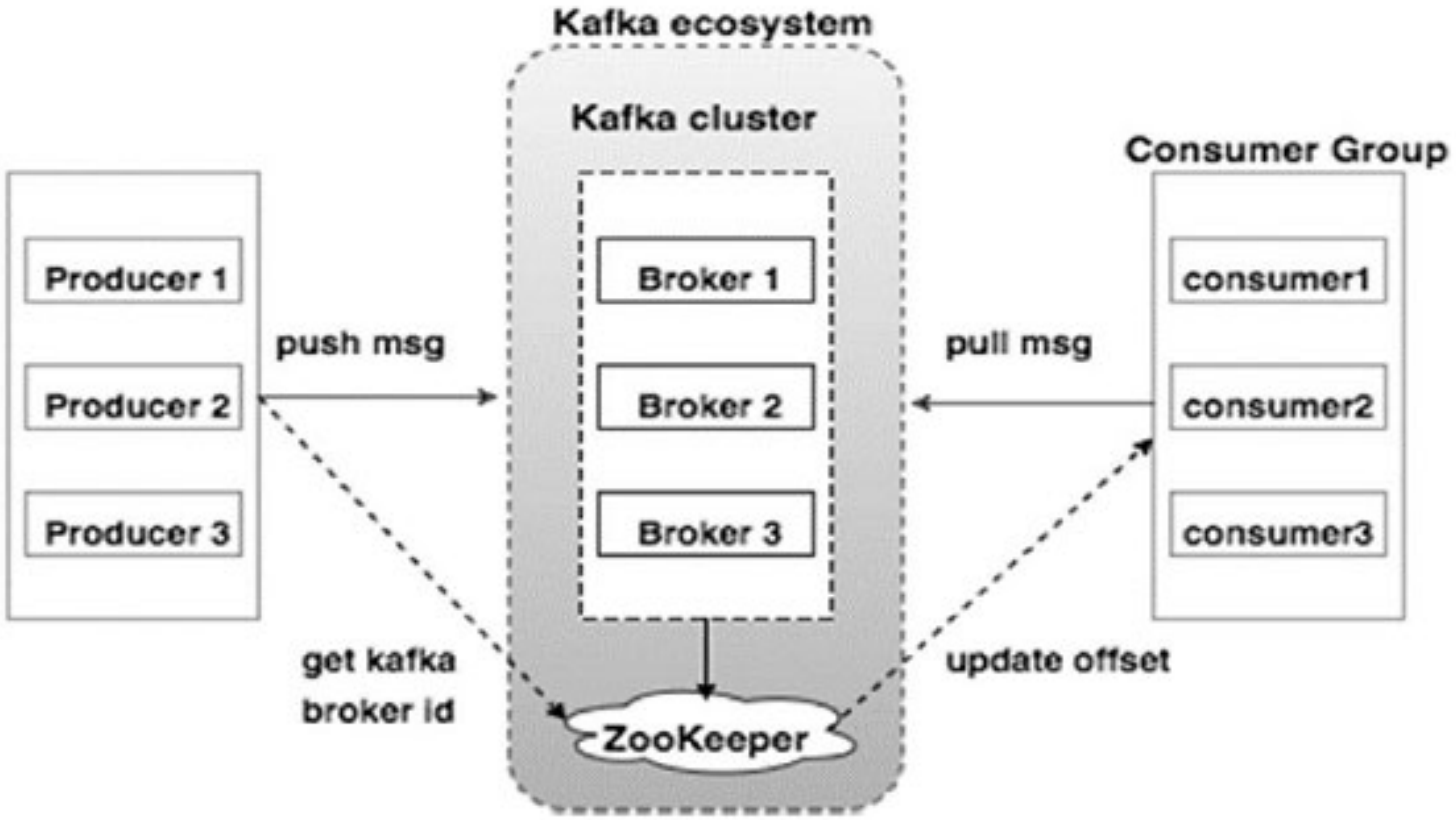
ZooKeeper



ZooKeeper service is mainly used to notify producer and consumer

- The presence of any new broker in the Kafka system or
- Failure of the broker in the Kafka system.

ZooKeeper



ZooKeeper service is mainly used to notify producer and consumer

- The presence of any new broker in the Kafka system or
- Failure of the broker in the Kafka system.

Apache Kafka + Spark Stream



https://spark.apache.org/docs/2.2.0/streaming-kafka-0-10-integration.html

A Real Time Application - Twitter Feeds

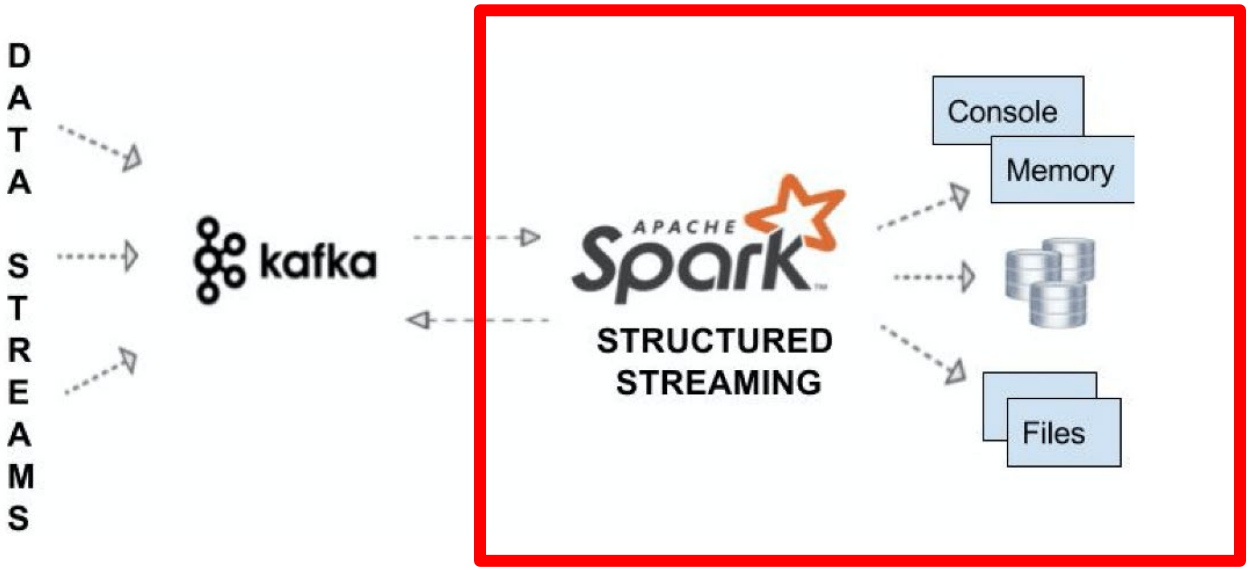
- First, we code a Kafka Producer to read Twitter feed and send to Kafka (Java)
- Second, we code Spark application integrated with Kafka to receive and process the stream (Scala)



https://databricks.com/blog/2017/04/04/real-time-end-to-end-integration-with-apache-kafka-in-apache-sparks-structured-streaming.html

A Real Time Application - Twitter Feeds

- First, we code a Kafka Producer to read Twitter feed and send to Kafka (Java)
- Second, we code Spark application integrated with Kafka to receive and process the stream (Scala)



https://databricks.com/blog/2017/04/04/real-time-end-to-end-integration-with-apache-kafka-in-apache-sparks-structured-streaming.html

Q7 - Regarding pub-sub messaging system, which statement is correct?

- The producer knows the destination of the message and posts the message directly to the consumer's queue.
- Only one consumer will get the message.
- Subscribers can decide to remain continuously active to receive messages or pause for a moment.
- Neither the publisher nor the subscriber know about each other. A good metaphor for this model is the anonymous bulletin board.

Q7 - Regarding pub-sub messaging system, which statement is correct?



The producer knows the destination of the message and posts the message directly to the consumer's queue.

- Only one consumer will get the message.
- Subscribers can decide to remain continuously active to receive messages or pause for a moment.
- Neither the publisher nor the subscriber know about each other. A good metaphor for this model is the anonymous bulletin board.

Q7 - Regarding pub-sub messaging system, which statement is correct?

- ✗ The producer knows the destination of the message and posts the message directly to the consumer's queue.
- ✗ Only one consumer will get the message.
- ○ Subscribers can decide to remain continuously active to receive messages or pause for a moment.
- ○ Neither the publisher nor the subscriber know about each other. A good metaphor for this model is the anonymous bulletin board.

Q7 - Regarding pub-sub messaging system, which statement is correct?

- ✗ The producer knows the destination of the message and posts the message directly to the consumer's queue.
- ✗ Only one consumer will get the message.
- ✗ Subscribers can decide to remain continuously active to receive messages or pause for a moment.
- ○ Neither the publisher nor the subscriber know about each other. A good metaphor for this model is the anonymous bulletin board.

Q7 - Regarding pub-sub messaging system, which statement is correct?

- ✗ The producer knows the destination of the message and posts the message directly to the consumer's queue.
- ✗ Only one consumer will get the message.
- ✗ Subscribers can decide to remain continuously active to receive messages or pause for a moment.
- ✓ Neither the publisher nor the subscriber know about each other. A good metaphor for this model is the anonymous bulletin board.

Q8 - What is the relationship between Kafka and Zookeeper?

- ZooKeeper services speed up the messaging efficiency by optimizing the channel to avoid congestion.
- ZooKeeper service is mainly used to notify producer and consumer: (i) the presence of any new broker in the Kafka system or (ii) the failure of the broker in the Kafka system.
- Kalfka and ZooKeeper are different software tools and they are irrelevant.
- Kafka brokers are stateless, so they use ZooKeeper to maintain their cluster state.

Q8 - What is the relationship between Kafka and Zookeeper?



ZooKeeper services speed up the messaging efficiency by optimizing the channel to avoid congestion.

- ZooKeeper service is mainly used to notify producer and consumer: (i) the presence of any new broker in the Kafka system or (ii) the failure of the broker in the Kafka system.
- Kalfka and ZooKeeper are different software tools and they are irrelevant.
- Kafka brokers are stateless, so they use ZooKeeper to maintain their cluster state.

Q8 - What is the relationship between Kafka and Zookeeper?



ZooKeeper services speed up the messaging efficiency by optimizing the channel to avoid congestion.



ZooKeeper service is mainly used to notify producer and consumer: (i) the presence of any new broker in the Kafka system or (ii) the failure of the broker in the Kafka system.

- ❑ Kalfka and ZooKeeper are different software tools and they are irrelevant.
- ❑ Kafka brokers are stateless, so they use ZooKeeper to maintain their cluster state.

Q8 - What is the relationship between Kafka and Zookeeper?

- ZooKeeper services speed up the messaging efficiency by optimizing the channel to avoid congestion.
- ZooKeeper service is mainly used to notify producer and consumer: (i) the presence of any new broker in the Kafka system or (ii) the failure of the broker in the Kafka system.
- Kalfka and ZooKeeper are different software tools and they are irrelevant.
- Kafka brokers are stateless, so they use ZooKeeper to maintain their cluster state.

Q8 - What is the relationship between Kafka and Zookeeper?



ZooKeeper services speed up the messaging efficiency by optimizing the channel to avoid congestion.



ZooKeeper service is mainly used to notify producer and consumer: (i) the presence of any new broker in the Kafka system or (ii) the failure of the broker in the Kafka system.



Kalfka and ZooKeeper are different software tools and they are irrelevant.



Kafka brokers are stateless, so they use ZooKeeper to maintain their cluster state.

Questions?